

DocSaarthi: AI Legal & Financial Document Assistant

Micro Project

Submitted in partial fulfillment of the requirements
for the Semester V Self-Learning Activity of

Natural Language Processing

(CMCOR1PE301.1)

for the award of the degree of

Bachelor of Technology

in

Computer Engineering

Submitted by

Akshata Naik (124BTCM1104)

Padmaja Shinde (124BTCM1211)

Under the Guidance of

Dr. Atul Haribhau Kachare

Assistant Professor



Department of Computer Engineering
Shah and Anchor Kutchhi Engineering College

(An Autonomous Institute Affiliated to University of Mumbai)

Mumbai – 400 088, Maharashtra, India

A.Y. 2026-2027



Mahavir Education Trust's

Shah & Anchor Kutchhi Engineering College

Autonomous Institute Affiliated to University of Mumbai
Bestowed with the University of Mumbai's Best College Honor A.Y. 2024-25
www.sakec.ac.in

CERTIFICATE

It is certified that the Micro-Project entitled

"DocSaarthi: AI Legal & Financial Document Assistant"

submitted by

Akshata Naik (124BTCM1104)

Padmaja Shinde (124BTCM1211)

has been carried out under my guidance during the Academic Year **A.Y. 2026-2027** as a partial fulfillment of the requirements for the **Semester V** Self-Learning Activity of the course **Natural Language Processing (CMCOR1PE301.1)** for the award of the degree of **Bachelor of Technology** in **Computer Engineering** at **Shah and Anchor Kutchhi Engineering College**.

The work presented in this report is original and has attained the standard expected for the Micro-Project conducted under the autonomous curriculum of the Institute.

The final review of the above-mentioned Micro-Project was held on:

Date: 12 July 2026

Dr. Atul Haribhau Kachare

Assistant Professor

Department of Computer Engineering

Shah and Anchor Kutchhi Engineering College

CANDIDATE'S DECLARATION

We hereby declare that the Micro-Project Report entitled

“DocSaarthi: AI Legal & Financial Document Assistant”

submitted in partial fulfillment of the requirements for the **Semester V** Self-Learning Activity of the course **Natural Language Processing (CMCOR1PE301.1)** for the award of the degree of **Bachelor of Technology in Computer Engineering**, submitted to the **Department of Computer Engineering, Shah and Anchor Kutchhi Engineering College**, is a bonafide record of our own work carried out during the Academic Year **A.Y. 2026-2027** under the guidance of **Dr. Atul Haribhau Kachare**.

We further declare that this work has not been submitted, either in full or in part, to any other University or Institute for the award of any degree, diploma, or any other academic qualification.

Name of the Student	PRN	Signature
Akshata Naik	124BTCM1104	
Padmaja Shinde	124BTCM1211	

Date: 12 July 2026

Place: Mumbai



Mahavir Education Trust's
Shah & Anchor Kutchhi Engineering College

Autonomous Institute Affiliated to University of Mumbai
Bestowed with the University of Mumbai's Best College Honor A.Y. 2024-25
www.sakec.ac.in

CERTIFICATE OF PLAGIARISM CHECK

CERTIFICATE OF PLAGIARISM CHECK	
Project Title	DocSaarthi: AI Legal & Financial Document Assistant
Project Type	Micro Project
Course	Natural Language Processing (CMCOR1PE301.1)
Academic Year	A.Y. 2026-2027
Name of Student(s)	Akshata Naik (124BTCM1104) Padmaja Shinde (124BTCM1211)
Name of Guide	Dr. Atul Haribhau Kachare
Designation	Assistant Professor
Department	Department of Computer Engineering
Similarity Index	6 %
Maximum Permissible Similarity	15%
AI Writing Index	3 %
Plagiarism Detection Tool	Turnitin
Date of Verification	12 July 2026
Student Signature(s)	Akshata Naik . . Padmaja Shinde

Guide Signature	Dr. Atul Haribhau Kachare
------------------------	---------------------------------

ACKNOWLEDGEMENT

We express our sincere gratitude to our guide, **Dr. Atul Haribhau Kachare, Assistant Professor, Department of Computer Engineering**, for providing continuous guidance, valuable suggestions, constructive feedback, and constant encouragement throughout the completion of this Micro Project undertaken as part of the course **Natural Language Processing (CMCOR1PE301.1)**.

We are grateful to **Dr. Bhavesh Patel**, Principal, **Shah and Anchor Kutchhi Engineering College**, for providing an excellent academic environment, infrastructure, and institutional support that facilitated the successful completion of this work.

We sincerely thank **Dr. Vidyullata Devmane**, Head of the **Department of Computer Engineering**, for continuous encouragement, motivation, and support throughout this project.

We also acknowledge the support and cooperation of all the faculty members and technical staff of the Department of Computer Engineering for providing the necessary facilities, resources, and technical assistance during the execution of the project.

We extend our heartfelt thanks to our project team members for their cooperation, coordination, and collaborative efforts, which greatly contributed to the successful completion of this Micro-Project.

Finally, we express our sincere gratitude to our parents, family members, and friends for their constant encouragement, moral support, and motivation throughout the development of this project.

Akshata Naik

Padmaja Shinde

Date: 12 July 2026

DEDICATION

We dedicate this Micro-Project to our beloved parents, whose unconditional love, constant encouragement, and unwavering support have been the foundation of our academic journey.

We also dedicate this work to our teachers and mentors, whose guidance, inspiration, and valuable knowledge have motivated us to learn, grow, and strive for excellence.

Finally, we dedicate this project to all those who have encouraged us to pursue knowledge with dedication, integrity, and perseverance.

Akshata Naik
Padmaja Shinde

ABSTRACT

Keywords: [Natural Language Processing (NLP), Legal and Financial Documents, English-Hindi Translation, Document Classification, Key Information Extraction, Bilingual Question Answering, IndicTrans2, Transformer-based Models.]

Contents

List of Figures	viii
------------------------	-------------

List of Tables	ix
-----------------------	-----------

1 Problem Identification	1
1.1 Introduction	1
1.1.1 Background	1
1.1.2 Domain Overview	2
1.1.3 Need for the Proposed NLP Solution	2
1.2 Problem Statement	3
1.3 Importance of the Problem	4
1.4 Project Objectives	4
1.5 Scope of the Project	5
1.5.1 In Scope	5
1.5.2 Out of Scope	5
1.6 Assumptions and Constraints	6
2 Literature Review	7
2.1 Introduction	7
2.2 Research Paper Review	8
2.2.1 IndicTrans2: Towards High-Quality and Accessible Machine Trans- lation Models for all 22 Scheduled Indian Languages	8
2.2.2 MuRIL: Multilingual Representations for Indian Languages	10
2.2.3 ILDC for CJPE: Indian Legal Documents Corpus for Court Judg- ment Prediction and Explanation	12
2.2.4 FinBERT: Financial Sentiment Analysis with Pre-trained Language Models	15
2.2.5 Named Entity Recognition in Indian Court Judgments	16
2.2.6 BERT: Pre-training of Deep Bidirectional Transformers for Lan- guage Understanding	18
2.3 Industrial Solution Review	21
2.3.1 Google Cloud Document AI	21
2.3.2 Microsoft Azure AI Document Intelligence	21

2.4	Comparative Analysis	21
2.5	Research Gap Analysis	23
2.6	Proposed Improvements	23
3	Dataset Collection	25
3.1	Introduction	25
3.2	Dataset Selection Criteria	25
3.3	Dataset Source	26
3.4	Dataset Description	26
3.5	Dataset Characteristics	27
3.5.1	Sample Records	29
3.6	Annotation Strategy	29
3.6.1	Existing Annotated Dataset	29
3.6.2	Self-created Dataset	29
3.7	Dataset Licensing and Permissions	31
3.8	Dataset Justification	31
3.9	Final Dataset and Data Split	32
3.9.1	Training Dataset	32
3.9.2	Validation Dataset	32
3.9.3	Testing Dataset	32
4	Data Preparation	34
4.1	Introduction	34
4.2	Data Cleaning	34
4.3	Tokenization	36
4.4	Stopword Processing	37
4.5	Morphological Processing	38
4.6	Exploratory Data Analysis (EDA)	39
4.6.1	Missing Value Analysis	40
4.6.2	Class Distribution	40
4.6.3	Vocabulary Analysis	41
4.6.4	Sentence Length Distribution	41
4.6.5	Most Frequent Tokens	41
4.6.6	N-gram Analysis	42
4.6.7	Word Cloud	43
4.6.8	Other Applicable Analyses	44
4.7	Preprocessing Workflow	44
4.8	Preprocessing Implementation	45
4.9	Before and After Preprocessing Analysis	45

5	NLP Pipeline Design	47
5.1	Introduction	47
5.2	Proposed System Architecture	47
5.3	Workflow Diagram	49
5.4	Feature Representation Strategy	50
5.4.1	Candidate Feature Representation Techniques	50
5.4.2	Selected Feature Representation	51
5.5	Model Selection Strategy	52
5.5.1	Candidate Models	52
5.5.2	Selected Model	53
5.6	System Requirements	54
5.6.1	Hardware Requirements	54
5.6.2	Software Requirements	55
5.7	Proposed Evaluation Strategy	55
5.7.1	Automatic Evaluation	55
5.7.2	Human Evaluation	56
5.8	Proposed and Implemented Pipeline	56
6	System Implementation	58
6.1	Introduction	58
6.2	Implementation Environment	58
6.2.1	Hardware Environment	58
6.2.2	Software Environment	59
6.3	Dataset Implementation	59
6.3.1	Dataset Loading	60
6.3.2	Dataset Usage	60
6.4	Preprocessing Implementation	60
6.4.1	PDF Text Extraction	61
6.4.2	Text Cleaning	61
6.4.3	Unicode Normalization	61
6.4.4	Document Chunking	61
6.5	Feature Representation Implementation	62
6.5.1	Lexical Representation	62
6.5.2	Bilingual Representation	62
6.5.3	Structured Information Representation	62
6.6	Model Implementation	63
6.6.1	Document Classification	63
6.6.2	Language Detection	63
6.6.3	Key Information Extraction	63
6.6.4	Question Answering	63

6.6.5	Translation	64
6.6.6	Optional AI Answer Generation	64
6.7	Training Configuration	64
6.8	Implementation Challenges and Solutions	65
6.9	Implemented System Architecture	66
7	Experimentation and Model Development	69
7.1	Introduction	69
7.2	Experimental Design	69
7.3	Baseline Model	70
7.4	Proposed Model	71
7.5	Feature Representation Experiments	73
7.5.1	Raw Text Representation	73
7.5.2	Normalized Token Representation	74
7.5.3	Chunk-Based Representation	74
7.5.4	Structured Information Representation	74
7.6	Model Comparison	75
7.7	Hyperparameter Experimentation	76
7.8	Experimental Observations	77
7.8.1	Observation 1: Importance of Preprocessing	77
7.8.2	Observation 2: Importance of Chunking	77
7.8.3	Observation 3: Importance of Page References	78
7.8.4	Observation 4: Bilingual Interaction	78
7.8.5	Observation 5: Source Grounding	78
7.8.6	Observation 6: External Service Dependency	78
7.8.7	Observation 7: Intent Dataset Usage	78
7.9	Selected Model and Configuration	78
7.10	Stage 3 Readiness	79
8	Individual Technical Contribution	81
8.1	Introduction	81
8.2	Individual Contribution Matrix	81
8.3	Individual Technical Tasks	83
8.3.1	Padmaja Shinde	83
8.3.2	Akshata Naik	84
8.4	Testing and Debugging Contribution	85
8.4.1	Padmaja Shinde	85
8.4.2	Akshata Naik	85
8.4.3	Joint Testing and Debugging	86
8.5	Problems Encountered and Solutions	87

8.6	Skill Acquisition and Technology Adoption	88
8.6.1	Padmaja Shinde	88
8.6.2	Akshata Naik	88
8.6.3	Technology Adoption	89
8.7	Individual Contribution Evidence	89
8.7.1	Evidence for Padmaja Shinde	90
8.7.2	Evidence for Akshata Naik	90
8.7.3	Project Artifacts as Evidence	91
9	Model Evaluation and Validation	93
9.1	Introduction	93
9.2	Evaluation Methodology	93
9.3	Evaluation Metrics	93
9.4	Quantitative Evaluation	93
9.5	Model Performance	94
9.6	Comparative Evaluation	94
9.7	Class-wise / Category-wise Evaluation	94
9.8	Human Evaluation (where applicable)	95
9.9	Validation Results	95
10	Error Analysis and Model Improvement	96
10.1	Introduction	96
10.2	Error Analysis Methodology	96
10.3	Error Categories	96
10.4	Representative Error Cases	96
10.5	Error Distribution	97
10.6	Analysis of Error Causes	98
10.7	Model Improvement	98
10.8	Before and After Improvement	99
10.9	Discussion of Improvement	99
11	Final System and Results	100
11.1	Introduction	100
11.2	Final System Architecture	100
11.3	Final End-to-End Workflow	101
11.4	Final System Implementation	101
11.5	Final Results	101
11.6	Sample Outputs	101
11.7	Results Discussion	102
11.8	Achievement of Project Objectives	102

11.9 Limitations of the Final System	102
12 SDG, Sustainability, Safety and Societal Impact	103
12.1 Introduction	103
12.2 SDG Alignment	103
12.3 Sustainability Considerations	103
12.4 Safety and Ethical Considerations	104
12.5 Societal Impact	104
12.6 Responsible Use and Limitations	104
13 Project Management and Team Contribution	105
13.1 Introduction	105
13.2 Work Distribution	106
13.3 Project Timeline and Milestones	106
13.4 Team Coordination	107
13.5 Testing and Quality Assurance	107
13.6 Version / Progress Tracking	107
13.7 Individual Contribution Summary	107
14 Conclusion and Future Scope	108
14.1 Introduction	108
14.2 Project Conclusion	108
14.3 Key Findings	108
14.4 Project Limitations	108
14.5 Future Scope	109
Bibliography	110
A AI Compliance	111
A.1 AI Tools Used	111
A.2 Nature of Assistance	111
A.3 Ethical Compliance Statement	112
B Plagiarism Report	113
C AI Writing Report	116
D Assessment Rubrics	119
E Supporting Evidence	122
E.1 Dataset Samples	122
E.2 Preprocessing Examples	122

E.3 Experimental Logs 122

E.4 Additional Results 122

E.5 Individual Contribution Evidence 123

List of Figures

4.1	Document_Type and Intent Class Distribution	40
4.2	Sentence Length Distribution (words per record)	41
4.3	Top 20 Most Frequent Tokens	42
4.4	Word Cloud of Dataset Vocabulary	43
4.5	Preprocessing Workflow	44
5.1	Proposed NLP System Architecture for DocSaarthi	48
5.2	Proposed NLP Workflow for DocSaarthi	49
6.1	Implemented System Architecture of DocSaarthi	67
7.1	Experimental Workflow of DocSaarthi	70
11.1	Final Implemented System Architecture	100

List of Tables

2.1	Literature Review of IndicTrans2	8
2.2	Literature Review of MuRIL	10
2.3	Literature Review of ILDC for CJPE	13
2.4	Literature Review of FinBERT	15
2.5	Literature Review of Named Entity Recognition in Indian Court Judgments	17
2.6	Literature Review of BERT	19
2.7	Comparative Analysis of Existing Research	22
3.1	Dataset Description	27
3.2	Dataset Characteristics	28
3.3	Sample Dataset Records	29
4.1	Data Cleaning Operations	35
4.2	Tokenization Example	37
4.3	Stopword Processing	38
4.4	Before and After Data Preprocessing Examples	39
4.5	Missing Value Analysis	40
4.6	Top 10 Most Frequent Bigrams	43
4.7	Representative Before and After Preprocessing Cases	45
5.3	Hardware Requirements	54
5.4	Software Requirements	55
6.1	Hardware Environment	59
6.2	Software Environment	59
6.3	Training and Processing Configuration	65
6.4	Implementation Challenges and Solutions	66
7.1	Baseline Configuration	71
7.2	Proposed Model Configuration	73
7.3	Feature Representation Experiments	74
7.4	Baseline and Proposed Model Comparison	75
7.5	Processing Parameter Experimentation	77

7.6	Selected Model and Configuration	79
8.1	Individual Contribution Matrix	82
8.2	Problems Encountered and Implemented Solutions	87
8.3	Technology Adoption	89
9.1	Review 3 Quantitative Results	94
9.2	Confusion Matrix for the Eight-Document Holdout	94
10.2	Distribution of Automated Validation Coverage	98
10.3	Before and After Improvement	99
13.1	Review 3 Work Distribution	106

Chapter 1

Problem Identification

1.1 Introduction

This chapter introduces the domain and context of the proposed project, DocSaarthi: AI Legal & Financial Document Assistant, an integrated Natural Language Processing (NLP) system for simplifying the understanding of legal and financial documents for common users. The chapter presents the background of the problem, defines the problem statement, discusses the importance of the problem, states the project objectives, defines the scope of the work, and lists the assumptions and constraints under which the project is carried out.

1.1.1 Background

Legal and financial documents such as service agreements, loan contracts, insurance policies, tax notices, and court proceedings form an integral part of everyday personal and organizational life. Traditionally, such documents are drafted using formal, domain-specific vocabulary, verbose sentence constructions, and clause-based structuring that make them difficult for a layperson to interpret without professional legal or financial assistance.

At present, an individual who receives such a document typically depends on a lawyer, chartered accountant, or financial advisor to explain its content, or spends considerable time manually reading and cross-referencing unfamiliar terms. This manual process is time-consuming, costly, and inaccessible to a large section of the population, particularly in a linguistically diverse country such as India, where a significant number of users are more comfortable reading regional languages such as Hindi rather than English.

Recent advances in NLP, particularly transformer-based language models, multilingual machine translation systems such as IndicTrans2, and Indian-language-aware classification models such as IndicBERT and MuRIL, have made it possible to build sys-

tems that can translate, classify, and answer questions over domain-specific text with reasonable accuracy. These developments motivate the design of an integrated assistant that brings together multiple NLP capabilities to address the document comprehension problem in a single pipeline.

1.1.2 Domain Overview

The proposed project operates at the intersection of two application domains: **Legal** and **Finance**, unified under the broader theme of **Indian Language Technologies**, since a core requirement of the system is English-to-Hindi translation and bilingual response generation.

- **Domain characteristics:** Legal documents (service agreements, confidentiality clauses, court proceedings) are characterised by formal clause-based structuring, obligation-bearing language, and fixed legal terminology. Financial documents (loan agreements, payment notices, insurance policies, tax statements) are characterised by numerical data such as amounts, dates, and durations embedded within descriptive sentences.
- **Existing workflow:** A user currently reads the document manually, or consults a domain expert, to identify the document type and extract relevant obligations, dates, and monetary values.
- **Current technologies:** Existing tools largely address translation, classification, or question answering independently, using general-purpose (non-domain-adapted) NLP models with limited support for Hindi.
- **Role of text data:** The text of the document is the sole source of information used by the system; no external knowledge base is consulted, and all classification and question-answering decisions are grounded strictly in the input document.

1.1.3 Need for the Proposed NLP Solution

The need for an NLP-based solution to this problem arises from the following considerations:

- A large volume of legal and financial text is generated and exchanged daily, ranging from employment contracts to loan documents, which cannot be manually reviewed at scale.
- Manual reading and interpretation of clause-heavy documents is time-consuming and error-prone for non-expert users.

- There is a clear need for automation that can classify a document, extract its salient information, and answer natural-language queries about it without manual intervention.
- A substantial proportion of Indian users are more comfortable understanding legal and financial content in Hindi than in English, creating a genuine multi-lingual requirement.
- Intelligent information extraction (identifying parties, amounts, durations, and obligations) reduces the cognitive load of reading a document in full.
- Automation enables faster processing and improves the overall user experience compared to consulting a human expert for routine document queries.

1.2 Problem Statement

Legal and financial documents are often lengthy and complex, containing domain-specific clauses, financial terminology, and formal legal phrasing that make them difficult for common users to understand without expert assistance. Individuals who receive such documents – for example, employees signing service agreements, borrowers reviewing loan contracts, or policyholders reading insurance terms – are directly affected, since misunderstanding a document’s obligations can lead to financial loss or legal disadvantage.

Existing NLP-based document-assistance tools typically address only a single task, such as machine translation, document classification, or question answering, in isolation. A user therefore has to rely on multiple independent tools to fully understand one document, and Hindi-language support across such tools remains limited. This project proposes an integrated NLP pipeline that accepts an English legal or financial document, translates it into Hindi, classifies it into the Legal or Financial category, extracts key information, and answers user questions in both English and Hindi using only the content of the uploaded document.

Example Format

“Given a lengthy English legal or financial document, how can a system automatically translate, classify, extract key information from, and answer natural-language questions about that document, in both English and Hindi, without requiring the user to consult multiple separate tools or a domain expert?”

1.3 Importance of the Problem

Solving this problem is important for the following reasons:

- **Academic importance:** The problem combines several core NLP sub-tasks – machine translation, text classification, named entity/key-information extraction, and question answering – within a single applied pipeline, making it a suitable case study for multi-task NLP system design.
- **Industrial relevance:** Legal-tech and fintech platforms are increasingly adopting AI-assisted document review, and an integrated bilingual assistant is directly relevant to such industry use cases.
- **Social impact:** Simplifying access to legal and financial understanding benefits users who lack access to legal or financial expertise, particularly first-time borrowers, small business owners, and salaried employees.
- **Economic impact:** Faster, low-cost document comprehension can reduce dependence on paid consultation for routine document queries.
- **Support for Indian languages:** The system directly supports Hindi, aligning with the broader national emphasis on multilingual digital access, including initiatives such as Digital India and Bhashini.
- **Sustainable Development Goals (SDGs):** The project contributes to SDG 10 (Reduced Inequalities) by improving equitable access to legal and financial information, and to SDG 9 (Industry, Innovation and Infrastructure) through the application of AI-based automation.

1.4 Project Objectives

1. To study the problem of legal and financial document understanding as a multi-task NLP problem.
2. To collect and analyse a suitable Hindi-language legal and financial text dataset for classification and question-answering experiments.
3. To perform text preprocessing, including cleaning, tokenization, stopword processing, and normalization, appropriate to the Hindi language.
4. To design an NLP pipeline that integrates English-to-Hindi translation, document classification, key information extraction, and question answering.

5. To select suitable feature representation techniques (e.g. transformer-based embeddings) for Hindi legal and financial text.
6. To identify appropriate models for document classification (Legal / Financial) and for context-based question answering.
7. To define suitable evaluation metrics for each pipeline stage (translation, classification, extraction, and question answering).
8. To design an interactive interface concept (Streamlit-based) through which a user can upload a document, view its translation and classification, and ask questions in natural language.

1.5 Scope of the Project

1.5.1 In Scope

- Collection and analysis of a labelled Legal/Financial Hindi text dataset (400 records, 8 intent sub-classes).
- Data preprocessing and exploratory data analysis of the collected dataset.
- Design of an NLP pipeline covering translation, document classification, key information extraction, and question answering.
- Selection and justification of feature representation and model choices for document classification.
- Design of an evaluation strategy for each pipeline component.
- Conceptual design of a Streamlit-based bilingual (English/Hindi) user interface.

1.5.2 Out of Scope

- Full-scale production deployment of the system.
- Cloud hosting and continuous integration/deployment pipelines.
- Real-time, large-volume document processing.
- Legal validity certification or provision of professional legal/financial advice.
- Support for languages other than English and Hindi.
- Continuous or online model retraining beyond the scope of this micro project.

1.6 Assumptions and Constraints

Assumptions

- The collected dataset labels (Document_Type and Intent) are correctly annotated.
- Required NLP libraries and pre-trained multilingual/Indic models are available and accessible.
- Sufficient computing resources are available for experimentation with transformer-based models.
- Users interact with the system in either English or Hindi and provide readable, digitally typed text (not scanned images).

Constraints

- Limited dataset size (400 records), which constrains the training of deep models from scratch and motivates the use of pre-trained models.
- Limited computational resources available for a semester-long micro project.
- Domain-specific legal and financial vocabulary that may not be fully covered by general-purpose pre-trained language models.
- Language-specific preprocessing challenges associated with Hindi, such as Unicode normalization and the absence of universally standard stopwords lists.
- Time limitations imposed by the micro-project schedule.

Chapter Summary

This chapter introduced the selected NLP problem of legal and financial document understanding, discussed its background and significance, defined the problem statement, outlined the project objectives and scope, and presented the assumptions and constraints of the proposed DocSaarthi system. The next chapter reviews the existing research literature on machine translation, document classification, and question answering, compares current approaches, identifies research gaps, and motivates the proposed solution.

Chapter 2

Literature Review

2.1 Introduction

A literature review provides a comprehensive understanding of existing research related to the selected NLP problem. It helps identify the current state of the art, datasets, feature engineering techniques, models, evaluation metrics, and research gaps. Since DocSaarthi integrates four distinct NLP sub-tasks – English-Hindi machine translation, Legal/Financial document classification, key information extraction, and question answering – the papers reviewed in this chapter are chosen to cover each of these sub-tasks individually, along with the foundational transformer architecture that underlies most modern NLP systems.

2.2 Research Paper Review

2.2.1 IndicTrans2: Towards High-Quality and Accessible Machine Translation Models for all 22 Scheduled Indian Languages

Table 2.1: Literature Review of IndicTrans2

Parameter	Description
Reference	J. Gala, P. A. Chitale, A. K. Raghavan, V. Gumma, S. Doddapaneni, A. Kumar M, J. A. Nawale, A. Sujatha, R. Puduppully, V. Raghavan, P. Kumar, M. M. Khapra, R. Dabre, and A. Kunchukuttan, "IndicTrans2: Towards High-Quality and Accessible Machine Translation Models for all 22 Scheduled Indian Languages," Transactions on Machine Learning Research, 2023.
Title	IndicTrans2: Towards High-Quality and Accessible Machine Translation Models for all 22 Scheduled Indian Languages
Authors	Jay Gala, Pranjal A. Chitale, A K Raghavan, Varun Gumma, et al. (AI4Bharat)
Year of Publication	2023
Publisher / Conference / Journal	Transactions on Machine Learning Research (TMLR)
Objective	To build a single high-quality, open-source machine translation model supporting translation between English and all 22 scheduled Indian languages.
Problem Addressed	Absence of large parallel training data, robust benchmarks, and open multilingual translation models covering all Indian languages.
NLP Task	Neural Machine Translation (English ↔ Indian languages, including Hindi)
Language(s)	English and 22 scheduled Indian languages (including Hindi)

Parameter	Description
Dataset	Bharat Parallel Corpus Collection (BPCC) – 230 million bitext pairs; IN22 n-way parallel benchmark
Annotation Method	Automatic mining supplemented with manually translated sentence pairs (644K sentences)
Data Preprocessing	Script unification, sentence-level alignment, deduplication, and denoising of mined parallel corpora
Feature Representation	Subword tokenization with a shared multilingual vocabulary across scripts
Model / Algorithm	Transformer-based encoder-decoder Neural Machine Translation model (IndicTrans2, and a distilled variant IndicTrans2-Dist)
Evaluation Metrics	BLEU, chrF++, COMET, and human evaluation
Key Findings	IndicTrans2 is the first open-source model to support all 22 scheduled Indian languages and performs at par with, or better than, existing commercial and open translation systems.
Advantages	• First single model covering all 22 scheduled Indian languages, including Hindi • Open-source with permissive licensing, enabling direct reuse • Distilled variant (IndicTrans2-Dist) reduces deployment cost while retaining performance

Parameter	Description
Limitations	• Performance still varies across low-resource languages • Domain-specific terminology (e.g., dense legal or financial clauses) is not explicitly targeted during training • Requires significant compute for full-scale fine-tuning
Possible Improvements	Domain-adaptive fine-tuning on legal and financial parallel corpora could further improve translation fidelity for clause-heavy sentences.

2.2.2 MuRIL: Multilingual Representations for Indian Languages

Table 2.2: Literature Review of MuRIL

Parameter	Description
Reference	S. Khanuja, D. Bansal, S. Mehtani, S. Khosla, A. Dey, B. Gopalan, D. K. Margam, P. Aggarwal, R. T. Nagipogu, S. Dave, S. Gupta, S. C. B. Gali, V. Subramanian, and P. Talukdar, "MuRIL: Multilingual Representations for Indian Languages," arXiv preprint arXiv:2103.10730, 2021.
Title	MuRIL: Multilingual Representations for Indian Languages
Authors	Simran Khanuja, Diksha Bansal, Sarvesh Mehtani, et al. (Google Research India)
Year of Publication	2021
Publisher / Conference / Journal	arXiv preprint (Google Research)

Parameter	Description
Objective	To build a BERT-based multilingual language model specifically optimized for Indian languages, including code-mixed and transliterated text.
Problem Addressed	General-purpose multilingual models such as mBERT under-represent Indian languages due to limited vocabulary and training data allocated to them.
NLP Task	Language Representation Learning, usable for downstream classification, NER, and QA
Language(s)	17 Indian languages and their transliterated (Latin-script) counterparts, including Hindi
Dataset	Wikipedia, Common Crawl, PMINDIA, and Dakshina corpora
Annotation Method	Self-supervised (no manual annotation required for pre-training)
Data Preprocessing	Translation and transliteration pair augmentation, whole-word masking
Feature Representation	Contextual sub-word embeddings (BERT-style Word-Piece tokenization)
Model / Algorithm	BERT (base and large) architecture pre-trained from scratch on Indian-language corpora
Evaluation Metrics	Accuracy and F1-score on the XTREME cross-lingual benchmark
Key Findings	MuRIL significantly outperforms mBERT on all evaluated tasks for Indian languages and handles transliterated (Romanized) text effectively.

Parameter	Description
Advantages	• Purpose-built for Indian languages, unlike general multilingual models • Handles both native-script and transliterated Hindi text • Freely available and directly fine-tunable for classification tasks
Limitations	• Not specifically trained on legal or financial domain text • Base model still requires task-specific fine-tuning and labelled data • Large model size increases inference latency compared to lightweight classifiers
Possible Improvements	Fine-tuning MuRIL on domain-specific Legal/Financial Hindi corpora, as done in this project, can improve classification accuracy over the base pretrained model.

2.2.3 ILDC for CJPE: Indian Legal Documents Corpus for Court Judgment Prediction and Explanation

Table 2.3: Literature Review of ILDC for CJPE

Parameter	Description
Reference	V. Malik, R. Sanjay, S. K. Nigam, K. Ghosh, S. K. Guha, A. Bhattacharya, and A. Modi, "ILDC for CJPE: Indian Legal Documents Corpus for Court Judgment Prediction and Explanation," in Proc. 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (ACL-IJCNLP), 2021, pp. 4046–4062.
Title	ILDC for CJPE: Indian Legal Documents Corpus for Court Judgment Prediction and Explanation
Authors	Vijit Malik, Rishabh Sanjay, Shubham Kumar Nigam, Kripabandhu Ghosh, Shouvik Kumar Guha, Arnab Bhattacharya, Ashutosh Modi
Year of Publication	2021
Publisher / Conference / Journal	Association for Computational Linguistics (ACL-IJCNLP)
Objective	To introduce a large annotated corpus of Indian legal documents and benchmark transformer models for judgment classification and explanation.
Problem Addressed	Lack of a large-scale, labelled Indian legal document corpus for training and evaluating legal NLP models.
NLP Task	Legal Document Classification (judgment prediction) and Explanation
Language(s)	English (Indian Supreme Court judgments)
Dataset	ILDC – over 35,000 Indian Supreme Court case documents
Annotation Method	Labels derived from actual case outcomes (accepted/rejected), with a manually annotated explainability subset
Data Preprocessing	Document cleaning, chunking of long documents to fit transformer input limits

Parameter	Description
Feature Representation	Transformer-based contextual embeddings (BERT, RoBERTa, XLNet, LegalBERT variants)
Model / Algorithm	Hierarchical transformer classifiers, fine-tuned BERT/LegalBERT models
Evaluation Metrics	Accuracy, Macro F1-score
Key Findings	Domain-adapted legal transformer models (e.g., LegalBERT/InLegalBERT-style pretraining) outperform general-purpose BERT on Indian legal document classification tasks.
Advantages	• Large-scale, real-world Indian legal document corpus • Establishes a strong benchmark for Indian legal document classification • Demonstrates the value of domain-specific pre-training for legal text
Limitations	• Restricted to English-language Supreme Court judgments; no Hindi coverage • Focused on judgment outcome prediction rather than general document-type classification • Very long documents pose input-length challenges for transformer models
Possible Improvements	Extending domain-adapted legal transformer pretraining to shorter, everyday legal documents (agreements, notices) and to Hindi text, as targeted in this project.

2.2.4 FinBERT: Financial Sentiment Analysis with Pre-trained Language Models

Table 2.4: Literature Review of FinBERT

Parameter	Description
Reference	D. Araci, "FinBERT: Financial Sentiment Analysis with Pre-trained Language Models," arXiv preprint arXiv:1908.10063, 2019.
Title	FinBERT: Financial Sentiment Analysis with Pre-trained Language Models
Authors	Dogu Tan Araci
Year of Publication	2019
Publisher / Conference / Journal	arXiv preprint (University of Amsterdam)
Objective	To adapt the BERT language model to the financial domain for improved performance on financial text classification tasks.
Problem Addressed	General-purpose language models underperform on financial text due to specialized domain vocabulary and limited labelled financial data.
NLP Task	Financial Document/Sentiment Classification
Language(s)	English
Dataset	Financial PhraseBank, FiQA Task-1 Sentiment Dataset
Annotation Method	Existing expert-annotated sentiment labels from Financial PhraseBank and FiQA
Data Preprocessing	Standard BERT tokenization (WordPiece), domain-specific further pre-training on financial corpora
Feature Representation	Contextual embeddings from BERT further pre-trained on financial text
Model / Algorithm	BERT fine-tuned with a classification head (FinBERT)
Evaluation Metrics	Accuracy, F1-score, Mean Squared Error (MSE), R^2

Parameter	Description
Key Findings	Domain-adaptive further pre-training on financial corpora improves classification accuracy substantially over vanilla BERT, even with a small labelled training set.
Advantages	• Demonstrates strong performance gains from financial domain pre-training • Performs well even with limited labelled data, relevant to this project's 400-record dataset • Openly available pretrained checkpoints
Limitations	• Restricted to English financial text; no multilingual or Hindi support • Focused on sentiment polarity rather than document-type or intent classification • Domain corpus limited to news/analyst text rather than contracts or loan agreements
Possible Improvements	The domain-adaptive pretraining strategy used in FinBERT motivates fine-tuning multilingual models (e.g., MuRIL) on Hindi financial document text for the proposed system.

2.2.5 Named Entity Recognition in Indian Court Judgments

Table 2.5: Literature Review of Named Entity Recognition in Indian Court Judgments

Parameter	Description
Reference	P. Kalamkar, A. Agarwal, A. Tiwari, S. Gupta, S. Karn, and V. Raghavan, "Named Entity Recognition in Indian Court Judgments," in Proc. Natural Legal Language Processing Workshop (NLLP), 2022, pp. 184–193.
Title	Named Entity Recognition in Indian Court Judgments
Authors	Prathamesh Kalamkar, Astha Agarwal, Aman Tiwari, Smita Gupta, Saurabh Karn, Vivek Raghavan
Year of Publication	2022
Publisher / Conference / Journal	Natural Legal Language Processing Workshop, Association for Computational Linguistics
Objective	To build an annotated corpus and baseline model for extracting fine-grained legal named entities from Indian court judgment text.
Problem Addressed	Legal named entities (parties, statutes, precedents, case numbers) are more fine-grained than generic entity types and are not well handled by general-purpose NER models.
NLP Task	Named Entity Recognition (Key Information Extraction)
Language(s)	English
Dataset	46,545 annotated legal named entities across 14 entity types, sourced from Indian court judgments
Annotation Method	Manual annotation across four review cycles by legal annotators
Data Preprocessing	Sentence segmentation, tokenization using spaCy pipeline
Feature Representation	Contextual token embeddings from transformer-based encoders
Model / Algorithm	Transition-based parser / transformer-based sequence labelling model, fine-tuned for entity tagging

Parameter	Description
Evaluation Metrics	Entity-level Precision, Recall, F1-score
Key Findings	Fine-grained, domain-specific entity schemas (e.g., PETITIONER, RESPONDENT, STATUTE, PRECEDENT) substantially improve the usefulness of extracted information for downstream legal applications compared to generic PERSON/ORG/DATE tags.
Advantages	• Provides a reusable schema of fine-grained legal entity types • Demonstrates that transformer-based sequence labelling generalizes well to legal text • Publicly released dataset and baseline model
Limitations	• Focused on court judgments rather than everyday contracts, agreements, or financial documents • English-only; no Hindi or multilingual entity extraction • Entity schema does not include financial entities such as loan amount or repayment period
Possible Improvements	Adapting a similar fine-grained entity schema to cover both legal (parties, obligations) and financial (amounts, dates, durations) entities in Hindi text, as required for this project.

2.2.6 BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

Table 2.6: Literature Review of BERT

Parameter	Description
Reference	J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in Proc. Conf. North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT), 2019, pp. 4171–4186.
Title	BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding
Authors	Jacob Devlin, Ming-Wei Chang, Kenton Lee, Kristina Toutanova
Year of Publication	2019
Publisher / Conference / Journal	North American Chapter of the Association for Computational Linguistics (NAACL-HLT)
Objective	To introduce a bidirectional transformer language model pre-trained on large unlabelled text and fine-tunable across a wide range of NLP tasks, including classification and question answering.
Problem Addressed	Prior language representation models were either unidirectional or shallow bidirectional, limiting their ability to capture full sentence context for downstream tasks.
NLP Task	General-purpose Language Representation; applicable to Classification, NER, and Question Answering
Language(s)	English (base model); later extended to multilingual variants
Dataset	BooksCorpus and English Wikipedia (pre-training); GLUE, SQuAD (fine-tuning benchmarks)
Annotation Method	Self-supervised pre-training (masked language modelling, next sentence prediction); supervised fine-tuning on labelled downstream datasets
Data Preprocessing	WordPiece sub-word tokenization

Parameter	Description
Feature Representation	Deep bidirectional contextual embeddings via multi-layer Transformer encoder
Model / Algorithm	Transformer encoder (BERT-Base: 12 layers, BERT-Large: 24 layers)
Evaluation Metrics	Accuracy on GLUE tasks; Exact Match (EM) and F1-score on SQuAD question answering
Key Findings	BERT achieves state-of-the-art results across eleven NLP benchmark tasks, and the same pre-trained encoder can be fine-tuned for classification, NER, and extractive question answering with minimal task-specific architecture changes.
Advantages	• Single pre-trained architecture reusable across classification, NER, and QA – directly relevant to DocSaarthi's multi-task pipeline • Strong empirical performance across diverse NLP tasks • Forms the architectural basis for domain- and language-specific variants such as MuRIL, IndicBERT, and FinBERT
Limitations	• Original model is English-only and not adapted to Indian languages or legal/financial domains • Fixed maximum input sequence length (512 tokens) limits use on long documents • Requires labelled fine-tuning data for each downstream task

Parameter	Description
Possible Improvements	Using a language- and domain-adapted descendant of BERT (e.g., MuRIL fine-tuned on Hindi legal/financial text) instead of the original English BERT, as proposed in this project.

2.3 Industrial Solution Review

2.3.1 Google Cloud Document AI

Google Cloud Document AI provides managed optical character recognition, document parsing, classification, and specialized processors. It demonstrates the industrial value of converting unstructured documents into machine-readable information. However, a cloud-first commercial workflow can introduce cost, data-governance, and connectivity concerns for users handling sensitive legal and financial documents.

2.3.2 Microsoft Azure AI Document Intelligence

Azure AI Document Intelligence provides prebuilt and custom extraction models, layout analysis, and integration with enterprise applications. Its strengths include mature deployment support and structured extraction. For the DocSaarthi use case, however, domain-specific bilingual question routing, local source-grounded answers, and an academic low-cost deployment remain application-level responsibilities.

2.4 Comparative Analysis

Table 2.7 compares the reviewed research papers across the NLP task, dataset, language, feature representation, model, evaluation strategy, strengths, and limitations.

Table 2.7: Comparative Analysis of Existing Research

Work	NLP Task	Dataset	Language	Features	Model	Evaluation	Strengths	Limitations
IndicTrans2 [1]	Translation	BPCC (230M pairs)	Eng. + 22 Indic	Subword embeddings	Transformer NMT	BLEU, chrF++, COMET	All-22-language coverage, open-source	No legal/financial domain tuning
MuRIL [2]	Multilingual Representation	Wikipedia, Common Crawl, Dakshina	17 Indic incl. Hindi	Contextual sub-word embeddings	BERT (base/large)	XTREME (Acc./F1)	Strong Indic + transliteration support	No legal/financial pretraining
ILDC/CJPE [3]	Legal Classification	35k Supreme Court cases	English	Transformer embeddings	LegalBERT	Accuracy, Macro-F1	Large legal benchmark	English-only, judgment-focused
FinBERT [4]	Financial Classification	Financial PhraseBank, FiQA	English	Domain-tuned BERT embeddings	Fine-tuned BERT	Accuracy, F1, MSE	Works well on small labelled data	English-only, sentiment-focused
Legal NER [5]	Key Information Extraction	46.5k legal entities	English	Contextual token embeddings	Transformer sequence tagger	Precision, Recall, F1	Fine-grained legal entity schema	No financial entities, English-only
BERT [6]	General LM / QA	BooksCorpus, Wikipedia, SQuAD	English	Deep bidirectional embeddings	Transformer encoder	GLUE Acc., SQuAD EM/F1	Reusable across tasks, strong baseline	Not domain/language adapted

2.5 Research Gap Analysis

The reviewed literature demonstrates strong individual progress in each of the four NLP sub-tasks required by DocSaarthi: high-quality English-Indic translation (Indic-Trans2), Indic-adapted language representation (MuRIL), legal document classification (ILDC/CJPE), financial text classification (FinBERT), legal key-information extraction (Legal NER), and the general transformer architecture underlying question answering (BERT). However, the following gaps remain:

- No reviewed work integrates translation, classification, extraction, and question answering into a single end-to-end pipeline for Indian users.
- Legal NLP research (ILDC, Legal NER) is almost exclusively English-only and focused on formal court judgments rather than everyday legal documents such as service agreements or confidentiality clauses.
- Financial NLP research (FinBERT) is similarly English-only and centred on sentiment analysis of news/analyst text rather than classification of financial documents such as loan or insurance contracts.
- None of the reviewed legal or financial entity-extraction approaches operate on Hindi text, despite Hindi being a primary language of everyday legal and financial communication in India.
- Bilingual (English + Hindi) answer generation for legal/financial document question answering is not addressed by any of the reviewed works.

2.6 Proposed Improvements

The identified gaps motivate an integrated, bilingual assistant rather than a single isolated NLP model. DocSaarthi improves accessibility by combining local PDF extraction, English-Hindi language handling, Legal/Financial classification, structured value extraction, chunk-level retrieval, page citations, and an explicit abstention response. The implemented prototype uses lightweight lexical and rule-based methods so that it can run on modest hardware. Transformer fine-tuning remains a future enhancement after a larger real-world, licensed corpus is available.

Chapter Summary

This chapter reviewed six research papers covering machine translation, multilingual language representation, legal document classification, financial text classification,

legal named entity recognition, and the foundational BERT transformer architecture. A comparative analysis of the reviewed works was presented, followed by an identification of research gaps – principally the absence of an integrated, Hindi-capable, multi-task pipeline for legal and financial document understanding – and a discussion of how the proposed DocSaarthi system addresses these gaps. The next chapter describes the dataset collected and used for the classification and question-answering components of the proposed system.

Chapter 3

Dataset Collection

3.1 Introduction

The quality of an NLP system largely depends on the quality, diversity, and suitability of the dataset used for training and evaluation. Dataset selection is therefore a critical stage in the development of any Natural Language Processing application. The selected dataset should adequately represent the target domain, language, and NLP task while ensuring sufficient quantity and quality of textual data.

This chapter describes the criteria used for selecting the DocSaarthi dataset, its source, its characteristics, the annotation methodology followed, licensing information, and the justification for its use in the proposed project. The dataset is used to train and evaluate the Document Classification and Key Information Extraction stages of the proposed pipeline.

3.2 Dataset Selection Criteria

The dataset satisfies the following selection criteria.

- **Relevance to the selected NLP task:** Every record is a Hindi legal or financial document text pre-labelled with a document type and an intent, directly matching the Document Classification and Key Information Extraction stages of the proposed pipeline.
- **Availability:** The dataset is self-created and locally available in CSV format, ensuring unrestricted access throughout the project.
- **Sufficient number of records:** The dataset contains 400 records, adequate for training and evaluating classical machine learning baselines and fine-tuning lightweight transformer classifiers within the scope of a micro project.

- **Appropriate language:** All text is in Hindi (Devanagari script), directly matching the target output language of the translation stage of the proposed system.
- **Availability of annotations:** Each record carries two labels – `Document_Type` (Legal / Financial) and `Intent` (8 sub-classes) – required for the classification stage.
- **Good data quality:** The dataset contains no missing values and no duplicate records (verified in Section 3.5).
- **Balanced class distribution:** The two primary classes (Legal and Financial) are perfectly balanced at 200 records each.
- **Appropriate for academic use:** As a self-created dataset, no external licensing restriction applies.

3.3 Dataset Source

The dataset used in this project, `DocSaarthi_Dataset.csv`, is a **self-created dataset**. It was synthetically generated by populating a small number of Hindi legal and financial document templates (service agreements, court proceeding notices, loan documents, insurance policies, tax notices, payment notices, and confidentiality clauses) with randomly varied entities – party names, organization names, dates, monetary amounts, and durations – to produce 400 distinct, realistic-looking legal and financial text records.

This approach was adopted because publicly available, freely licensed, sentence-level Hindi legal and financial text corpora with document-type and intent labels are scarce, and it allowed full control over class balance across both the `Document_Type` and `Intent` labels. The self-created nature of the dataset is disclosed transparently, consistent with the annotation and licensing discussion in Sections 3.6 and 3.7.

- **Dataset file:** `DocSaarthi_Dataset.csv`
- **Access date:** As used for this micro project, Academic Year 2026–2027

3.4 Dataset Description

Table 3.1 summarizes the selected dataset.

Table 3.1: Dataset Description

Parameter	Description
Dataset Name	DocSaarthi_Dataset
Source	Self-created (synthetically generated from Hindi legal/financial document templates)
URL	Not applicable (locally created dataset)
Domain	Legal and Financial documents
NLP Task	Document Classification (Legal/Financial), Intent Classification, Key Information Extraction
Language(s)	Hindi (Devanagari script)
File Format	CSV
Dataset Size	400 records, 4 columns (ID, Text, Document_Type, Intent)
Training Split	280 records (70%) – proposed split, to be applied during implementation
Validation Split	60 records (15%) – proposed split, to be applied during implementation
Testing Split	60 records (15%) – proposed split, to be applied during implementation
License	Self-created for academic use; no third-party licensing restrictions
Version	v1.0
Date Accessed	Academic Year 2026–2027

3.5 Dataset Characteristics

Table 3.2 presents the important characteristics of the dataset, computed directly from `DocSaarthi_Dataset.csv`.

Table 3.2: Dataset Characteristics

Characteristic	Observation
Number of Records	400
Number of Classes (Document_Type)	2 (Legal, Financial)
Number of Classes (Intent)	8 (Agreement, Court Proceeding, Confidentiality, Finance, Loan, Payment, Tax, Insurance)
Class Labels (Document_Type distribution)	Legal: 200 (50.0%); Financial: 200 (50.0%)
Class Labels (Intent distribution)	Agreement: 70; Court Proceeding: 65; Confidentiality: 65; Finance: 40; Loan: 40; Payment: 40; Tax: 40; Insurance: 40
Average Sentence/Record Length (words)	46.9 words
Maximum Record Length (words)	71 words
Minimum Record Length (words)	30 words
Average Record Length (characters)	262.1 characters
Vocabulary Size	1,196 unique whitespace-delimited Hindi tokens
Language Distribution	100% Hindi (Devanagari script)
Missing Values	0 (no null values in any column)
Duplicate Records	0 (no duplicate Text entries)
Special Characters	Devanagari punctuation (।), currency symbol (₹), numerals, forward slashes in dates
Encoding Format	UTF-8

3.5.1 Sample Records

Table 3.3 shows representative records from the dataset, one for each of four Intent categories.

Table 3.3: Sample Dataset Records

Record ID	Text	Label
1	यह अनुबंध यह सेवा अनुबंध दनांक 23/05/2025 को आर्यन ट्रेडर्स तथा नेहा शर्मा के बीच संपन्न हुआ। अनुबंध की अवधि 29 माह निर्धारित की गई है। कर्मचारी को ₹31717 मासिक पारिश्रमिक दिया जाएगा। ...	Legal / Agreement
71	माननीय न्यायालय ने वाद संख्या 1757/2026 में कविता पाटलि बनाम कविता गुप्ता का मामला जिला न्यायालय में विचाराधीन है। अगली सुनवाई 08/09/2025 को निर्धारित की गई है। ...	Legal / Court Proceeding
202	ऋण दस्तावेज के अनुसार भारतीय स्टेट बैंक ने अजय पाटलि को ₹564670 का ऋण 14/12/2025 को स्वीकृत किया। वार्षिक ब्याज दर 9.83% है। मासिक ईएमआई ₹10509 निर्धारित की गई है। ...	Financial / Loan
205	कविता वर्मा ने एचडीएफसी लाइफ की बीमा पॉलिसी खरीदी। बीमा कवरेज ₹2647217 है तथा वार्षिक प्रीमियम ₹17928 है। पॉलिसी 30/05/2025 से प्रभावी होगी ...	Financial / Insurance

3.6 Annotation Strategy

3.6.1 Existing Annotated Dataset

Not applicable, since the dataset used in this project is self-created (see Section 3.6.2) rather than sourced from an already-annotated third-party corpus.

3.6.2 Self-created Dataset

Annotation Guidelines

Each record was generated from one of seven fixed Hindi document templates (service agreement, court proceeding notice, loan document, insurance policy, tax notice,

payment notice, confidentiality clause), where each template is intrinsically associated with exactly one `Document_Type` and one `Intent` label. Populating a template with randomized entity values (party names, organization names, dates, amounts, and durations) therefore produces a record whose label is deterministically correct by construction.

Label Definitions

- **Legal – Agreement:** Service agreements specifying parties, duration, and remuneration.
- **Legal – Court Proceeding:** Notices describing case numbers, parties, and hearing dates.
- **Legal – Confidentiality:** Clauses specifying non-disclosure obligations between parties.
- **Financial – Finance:** General financial statements or notices.
- **Financial – Loan:** Loan sanction documents specifying lender, borrower, amount, and interest rate.
- **Financial – Payment:** Payment due or payment confirmation notices.
- **Financial – Tax:** Tax-related notices or statements.
- **Financial – Insurance:** Insurance policy documents specifying coverage and premium.

Annotation Process

- Number of annotators: Not applicable (labels assigned programmatically and deterministically at the time of template-based generation)
- Annotation tool used: Custom Python-based template-filling script
- Review process: Manual spot-checking of a random sample of generated records to confirm label correctness and text readability

Quality Checking

- Random sampling and manual review of generated records across all eight `Intent` categories
- Programmatic verification of zero missing values and zero duplicate `Text` entries

Inter-Annotator Agreement

Not applicable, as labels are deterministically assigned by the template-generation process rather than by independent human annotators.

3.7 Dataset Licensing and Permissions

- **License type:** Self-created for academic use; no external license applies.
- **Terms of use:** The dataset is used solely for the academic purposes of this Micro Project.
- **Citation requirements:** None, as the dataset is not derived from a third-party publication.
- **Redistribution policy:** The dataset may be shared for academic evaluation of this project.
- **Privacy considerations:** All party names, organization names, and case numbers appearing in the dataset are synthetically generated and do not correspond to real individuals, companies, or actual court cases.
- **Ethical considerations:** As the dataset is fully synthetic, no real personal or financial data is exposed, avoiding privacy and confidentiality concerns associated with real legal or financial documents.

3.8 Dataset Justification

The selected dataset is suitable for the proposed project for the following reasons:

- **Appropriate size:** 400 records provide a workable dataset for training and evaluating lightweight classifiers and fine-tuning transformer models within the scope of a micro project.
- **Relevant domain:** The dataset spans exactly the two domains – Legal and Financial – targeted by the proposed system.
- **Suitable language:** All text is in Hindi, matching the target language of the translation and classification stages.
- **Balanced classes:** A perfectly balanced 200/200 Document_Type split, and reasonably balanced Intent classes (40–70 records each), reduce the risk of class-imbalance bias during model training.

- **High data quality:** No missing values or duplicate records were found during characteristic analysis.
- **Compatibility with the selected NLP task:** The dataset directly supports supervised training of the Document Classification module and provides realistic sentence structures (dates, monetary values, named entities) for evaluating the Key Information Extraction module.

3.9 Final Dataset and Data Split

The final intent dataset contains 400 Hindi records across nine intent labels. In the implemented prototype it is loaded as a complete reference set for query-intent signals; it is not used as evidence for answering questions about an uploaded document. For reproducible supervised experiments, the recommended stratified split is 70:15:15, with a fixed random seed and no template instance appearing in more than one partition.

3.9.1 Training Dataset

The proposed training partition contains 280 records (70%). It is intended for fitting future statistical or transformer-based intent classifiers.

3.9.2 Validation Dataset

The proposed validation partition contains 60 records (15%). It is reserved for threshold selection and model configuration, not final reporting.

3.9.3 Testing Dataset

The proposed testing partition contains 60 records (15%). It is held out for final intent-classification evaluation. Separately, the current Review 3 system evaluation uses eight labelled bilingual PDF documents to assess document-domain classification and language detection.

Chapter Summary

This chapter presented the dataset selected for the proposed DocSaarthi NLP project: a self-created, 400-record, template-generated Hindi legal and financial text corpus with balanced Document_Type and Intent labels. The dataset source, description,

characteristics, annotation strategy, licensing information, and suitability for the selected NLP task were discussed. A well-documented dataset provides the foundation for effective preprocessing and model development. The next chapter describes the preprocessing techniques and exploratory data analysis performed on the collected dataset.

Chapter 4

Data Preparation

4.1 Introduction

Raw textual data collected from various sources often contains inconsistencies, noise, redundant information, and formatting variations that may adversely affect the performance of Natural Language Processing (NLP) models. Data preparation is therefore an essential step in the NLP pipeline to improve data quality and ensure compatibility with feature extraction and machine learning algorithms.

The preprocessing techniques selected for this project are based on the requirements of the Legal/Financial document classification and key-information-extraction tasks and the characteristics of the Hindi-language dataset described in Chapter 3. Only the preprocessing operations applicable to the selected task are performed. Each preprocessing step is justified based on its contribution to improving model performance.

4.2 Data Cleaning

Data cleaning aims to remove unwanted information while preserving the semantic meaning of the text. As shown in Chapter 3, the raw dataset already contains no missing values, no duplicate records, and no HTML tags, emojis, or URLs (it is a clean, template-generated corpus of well-formed Hindi sentences). The cleaning operations below were therefore evaluated and applied selectively, only where relevant.

Table 4.1: Data Cleaning Operations

Operation	Applied (Yes/No)	Justification
Remove Duplicate Records	No (none found)	Verified in Chapter 3; 0 duplicate Text entries exist.
Remove Null Values	No (none found)	Verified in Chapter 3; 0 missing values across all columns.
Remove URLs	No	Legal/financial document text does not contain URLs.
Remove HTML Tags	No	Dataset is plain text, not scraped from web markup.
Remove Emojis	No	Formal legal/financial text does not contain emojis.
Remove Punctuation	Partial	Devanagari full stop (।) and standard punctuation are retained, as they carry sentence-boundary information needed for translation and QA; only extraneous symbols would be stripped.
Remove Special Characters	No	The currency symbol (₹) and % sign are financially meaningful and are retained for key information extraction.
Remove Numbers	No	Dates, amounts, durations, and case numbers are critical extraction targets and must be preserved.
Unicode Normalization	Yes	Applied (NFC normalization) to ensure consistent encoding of Devanagari conjuncts and matras across all records.
Case Conversion	Not applicable	Devanagari script has no case distinction.

Operation	Applied (Yes/No)	Justification
Whitespace Normalization	Yes	Extra/leading/trailing whitespace is normalized to ensure consistent tokenization.
Language-specific Cleaning	Yes	Normalization of Devanagari punctuation (।) usage for consistent sentence segmentation.

4.3 Tokenization

Tokenization divides text into meaningful units for further processing. For Hindi, whitespace-based word tokenization is a reasonable first approximation since Devanagari text is space-delimited at the word level, but subword tokenization is preferred for transformer-based models to handle Hindi's morphological richness (inflections for gender, number, and case).

Possible techniques considered:

- Word Tokenization (whitespace-based)
- Sentence Tokenization (Devanagari पूर्ण वरिम "।" as delimiter)
- Subword Tokenization
- WordPiece (used internally by BERT/MuRIL-family models)
- SentencePiece (used internally by IndicTrans2)

For exploratory analysis in this chapter, whitespace-based word tokenization is used (yielding a vocabulary of 1,196 unique tokens, as reported in Chapter 3). For the eventual classification and QA models, the WordPiece/SentencePiece tokenizer bundled with the selected pretrained model (MuRIL/IndicTrans2) will be used instead, since sub-word tokenization better handles unseen inflected forms in Hindi.

Example

Table 4.2: Tokenization Example

Original Sentence	ऋण दस्तावेज़ के अनुसार भारतीय स्टेट बैंक ने अजय पाटलि को ₹564670 का ऋण 14/12/2025 को स्वीकृत किया।
Tokenized Output	[ऋण, दस्तावेज़, के, अनुसार, भारतीय, स्टेट, बैंक, ने, अजय, पाटलि, को, ₹564670, का, ऋण, 14/12/2025, को, स्वीकृत, किया।]

4.4 Stopword Processing

Common Hindi function words such as की, के, का, को, में, है, ने, तथा dominate the token-frequency list (Section 4.5.4). Whether these should be removed depends on the downstream task:

- For **document/intent classification** using TF-IDF or classical ML models, removing high-frequency stopwords can help the model focus on discriminative content words (e.g., ऋण, बीमा, न्यायालय, अनुबंध).
- For **transformer-based classification and question answering**, stopwords are **retained**, since contextual models such as MuRIL rely on complete sentence structure – and stopwords are essential for phrasing questions and answers naturally in Hindi.

Since the proposed pipeline (Chapter 5) uses a fine-tuned transformer for both classification and QA, stopwords are retained for the primary pipeline; stopword removal is treated as an optional preprocessing branch for classical ML baseline comparisons only.

Table 4.3: Stopword Processing

Stopwords Removed	No (retained for transformer-based pipeline); optional for classical ML baselines
Stopword Source	Indic NLP Library Hindi stopwords list (for the optional classical-ML branch)
Reason	Contextual transformer models require complete sentence structure; stopwords carry grammatical information needed for QA and translation.

4.5 Morphological Processing

Hindi is a morphologically rich, inflected language (verbs and nouns inflect for gender, number, case, and tense), so aggressive English-style stemming is not directly applicable.

Possible techniques include:

- Stemming
- Lemmatization

Possible algorithms/tools:

- Indic NLP Library (rule-based Hindi stemmer)
- iNLTK
- Stanza (with Hindi UD models, for lemmatization)

Selected Technique

Light rule-based Hindi **stemming** using the Indic NLP Library, applied only for the optional classical ML baseline (TF-IDF + Logistic Regression/SVM). No stemming is applied before transformer-based fine-tuning, since sub-word tokenizers (WordPiece/SentencePiece) already learn morphological regularities directly from data.

Justification

Since the final proposed model (Chapter 5) is a fine-tuned transformer, morphological normalization is delegated to the sub-word tokenizer rather than applied manually. Manual stemming is retained only as a preprocessing option for lightweight classical ML baselines used for comparison.

Before and After Example

Table 4.4: Before and After Data Preprocessing Examples

Preprocessing Technique	Before	After
Whitespace Normalization	"अनुबंध की अवधि"	"अनुबंध की अवधि"
Case Conversion	Not applicable (Devanagari)	Not applicable (Devanagari)
Punctuation Retained	कर्मचारी को ₹31717 मासिक पारिश्रमिक दिया जाएगा।	कर्मचारी को ₹31717 मासिक पारिश्रमिक दिया जाएगा। (I retained as sentence boundary marker)
Word Tokenization	अनुबंध की अवधि 29 माह निर्धारित की गई है।	[अनुबंध, की, अवधि, 29, माह, निर्धारित, की, गई, है।]
Stopword Removal (optional ML branch)	अनुबंध की अवधि 29 माह निर्धारित की गई है।	अनुबंध अवधि 29 माह निर्धारित

4.6 Exploratory Data Analysis (EDA)

The following analyses, applicable to the DocSaarthi dataset, were performed directly on `DocSaarthi_Dataset.csv`.

- Missing Value Analysis
- Class Distribution (Document_Type and Intent)
- Vocabulary Size Analysis
- Sentence Length Distribution
- Most Frequent Tokens

- N-gram (Bigram) Analysis
- Word Cloud

4.6.1 Missing Value Analysis

Table 4.5 presents the missing value analysis of the dataset. It can be observed that all four columns contain zero missing values, indicating that the dataset is complete and does not require any imputation before preprocessing.

Table 4.5: Missing Value Analysis

Column Name	Missing Values	Percentage (%)
ID	0	0.00
Text	0	0.00
Document_Type	0	0.00
Intent	0	0.00

Observation: The dataset is complete, with no null values present in any column. Therefore, no missing value imputation was required before applying the NLP preprocessing pipeline.

4.6.2 Class Distribution

Figure 4.1 shows the distribution of the Document_Type and Intent labels.

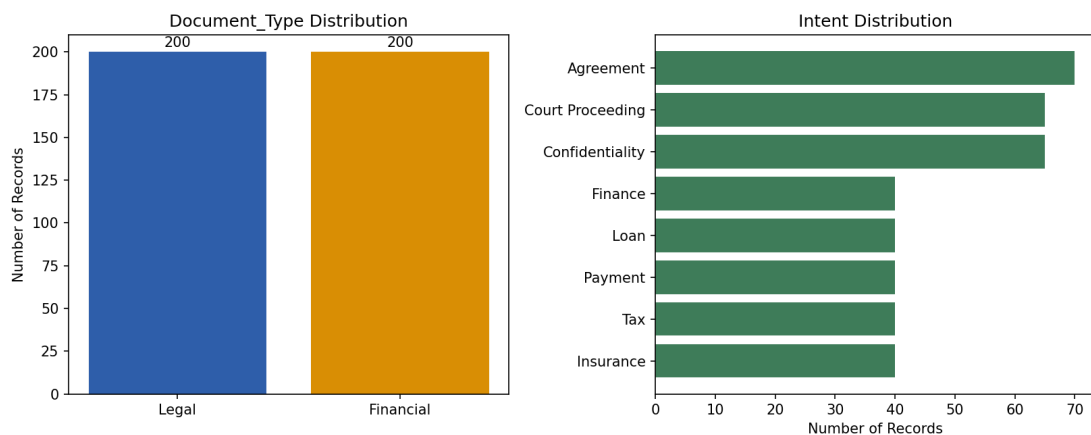


Figure 4.1: Document_Type and Intent Class Distribution

Observation: Document_Type is perfectly balanced (200 Legal / 200 Financial). Intent is moderately imbalanced: Agreement (70), Court Proceeding (65), and Confidentiality (65) are more frequent than the four Financial intents (40 each), reflecting the template-generation design. This imbalance should be accounted for (e.g., via class-weighted loss) when training the Intent classifier.

4.6.3 Vocabulary Analysis

The dataset contains **1,196 unique whitespace-delimited tokens** across 400 records, giving a type-token ratio of approximately $1,196 / 18,764 \approx 0.064$, indicating a fairly repetitive, template-driven vocabulary – expected given the synthetic generation process described in Chapter 3.

4.6.4 Sentence Length Distribution

Figure 4.2 shows the histogram of record lengths measured in words.

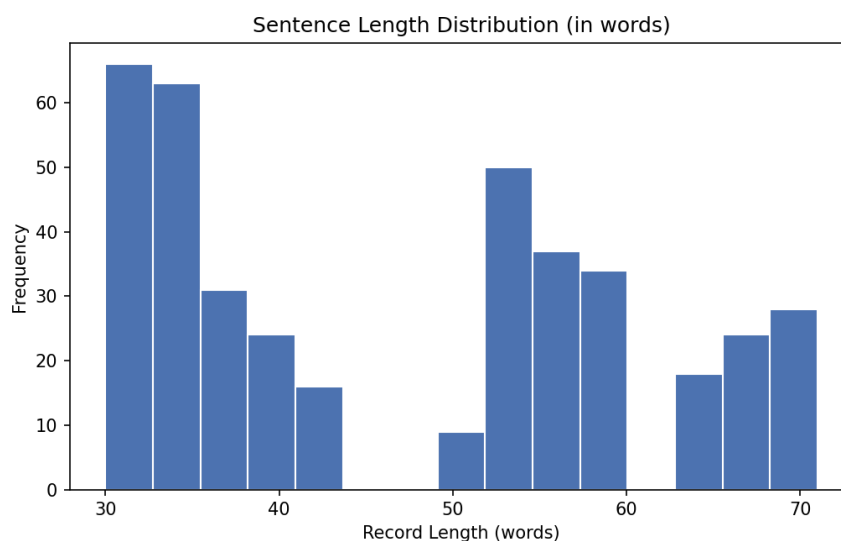


Figure 4.2: Sentence Length Distribution (words per record)

Observation: Record lengths range from 30 to 71 words, with a mean of 46.9 words and a roughly bimodal spread reflecting the two document categories (shorter Financial notices versus longer Legal clauses). This length range is well within the input limits of standard transformer models (512 tokens), so no truncation is required.

4.6.5 Most Frequent Tokens

Figure 4.3 shows the twenty most frequent tokens in the dataset.

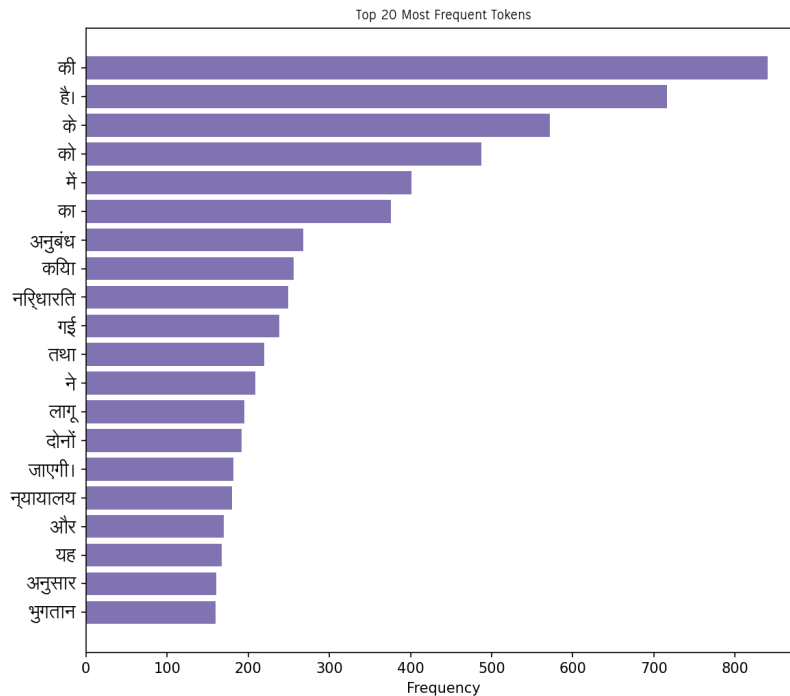


Figure 4.3: Top 20 Most Frequent Tokens

Observation: The most frequent tokens (की, है, के, को, में, का) are Hindi function words (postpositions and copulas), as expected. Content-bearing domain tokens such as अनुबंध (agreement/contract), निर्धारति (fixed/determined), न्यायालय (court), and भुगतान (payment) also appear among the top 20, confirming that the dataset carries clear domain signal beyond generic function words.

4.6.6 N-gram Analysis

Table 4.6 presents the ten most frequent bigrams found in the dataset.

Table 4.6: Top 10 Most Frequent Bigrams

Bigram	Frequency
गई है।	221
नरिधारति की	215
की गई	215
के अनुसार	161
की जाएगी।	156
कयिा गया।	136
के बीच	135
दोनों पक्षों	122
कयिा गया	112
गया है।	112

Observation: The dominant bigrams (“नरिधारति की”, “की गई”, “के अनुसार”) are formal legal/financial phrasing patterns (“...is determined”, “...as per...”), confirming the formulaic, clause-based writing style typical of legal and financial documents.

4.6.7 Word Cloud

Figure 4.4 presents a word cloud generated from the complete dataset vocabulary.

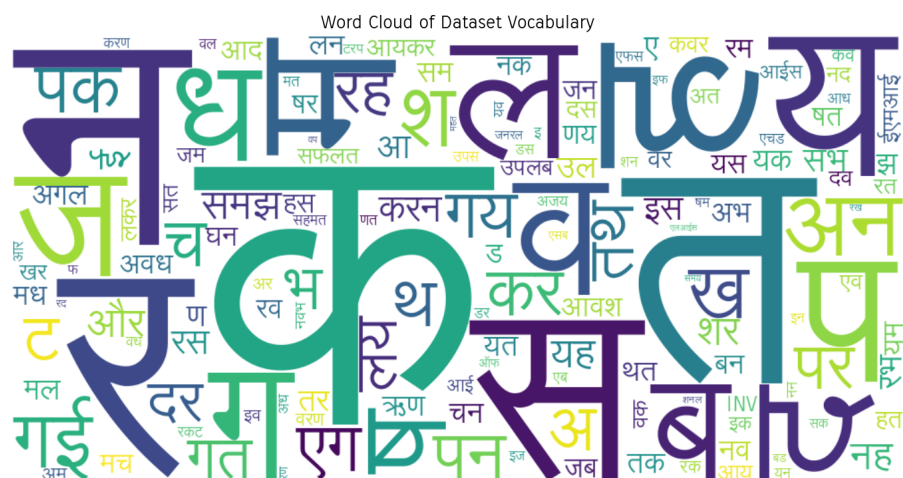


Figure 4.4: Word Cloud of Dataset Vocabulary

Observation: The word cloud visually reinforces the frequency analysis, with domain terms such as अनुबंध, न्यायालय, भुगतान, and नरिधारति appearing prominently alongside common Hindi function words.

4.6.8 Other Applicable Analyses

Character-level and language distribution analyses were reported in Table 3.5 (Chapter 3): the dataset is 100% Devanagari-script Hindi text with an average record length of 262 characters, and required no out-of-vocabulary (OOV) analysis since it is a closed, self-generated corpus.

4.7 Preprocessing Workflow

Figure 4.5 illustrates the complete preprocessing pipeline used in the proposed project.

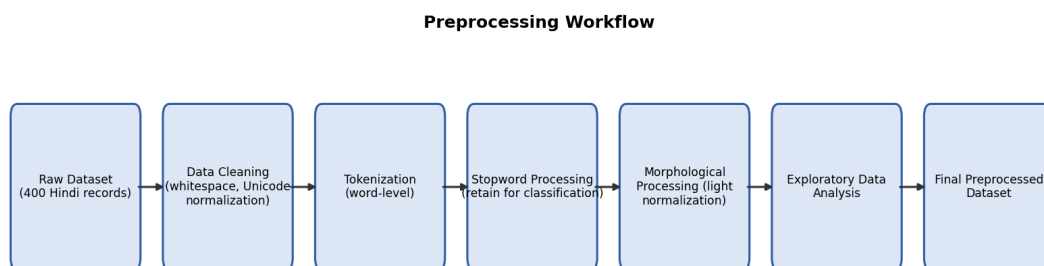


Figure 4.5: Preprocessing Workflow

The workflow illustrates:

- Raw Dataset (400 Hindi legal/financial records)
- Data Cleaning (Unicode and whitespace normalization)
- Tokenization (word-level for EDA; sub-word for model input)
- Stopword Processing (retained for transformer pipeline; optional removal for classical ML baseline)
- Morphological Processing (delegated to sub-word tokenizer; optional stemming for baseline)
- Exploratory Data Analysis (class distribution, vocabulary, sentence length, n-grams)
- Final Preprocessed Dataset (ready for feature representation and model training, Chapter 5)

4.8 Preprocessing Implementation

The implemented Node.js pipeline normalizes Unicode text, repairs whitespace and PDF line-break artefacts, preserves Devanagari combining marks, divides pages into evidence chunks, and retains each chunk's page number. Stopwords are not removed from the source evidence because legal negation and function words can change meaning. Translation is deferred until requested so that document upload remains fast and independent of a translation service.

4.9 Before and After Preprocessing Analysis

Table 4.7: Representative Before and After Preprocessing Cases

Before	After
Repeated spaces and isolated PDF line breaks	Normalized spacing with sentence continuity restored
Mixed Unicode forms in Hindi text	Unicode-normalized Devanagari with combining marks preserved
Complete document treated as one search unit	Page-linked chunks suitable for focused retrieval
Translation coupled to upload	Original source stored first; translation performed only on demand

The analysis shows that preprocessing improves retrieval consistency without rewriting the authoritative source text. Numeric values, dates, percentages, names, and page references remain available for answer verification.

Chapter Summary

This chapter presented the preprocessing techniques applied to the DocSaarthi dataset before model development. Data cleaning confirmed the dataset was already free of missing values, duplicates, and noise; Unicode and whitespace normalization were nonetheless applied. Tokenization, stopwords processing, and morphological processing were evaluated and selectively applied based on whether the downstream model is a transformer (stopwords/morphology retained, delegated to the sub-word tokenizer) or a classical ML baseline (stopwords/stemming applied). Exploratory data

analysis revealed a perfectly balanced Document_Type split, a moderately imbalanced Intent distribution, a compact 1,196-word vocabulary reflecting the dataset's template-driven generation, and clause-based bigram patterns typical of legal/financial writing. The resulting preprocessed dataset forms the input for the feature representation and model design discussed in the next chapter.

Chapter 5

NLP Pipeline Design

5.1 Introduction

This chapter presents the proposed Natural Language Processing (NLP) pipeline designed for DocSaarthi: AI Legal & Financial Document Assistant. It describes the overall system architecture, the end-to-end workflow, the feature representation strategy, the model selection strategy, the system requirements, and the proposed evaluation methodology. The design integrates four NLP sub-tasks – English-to-Hindi machine translation, Legal/Financial document classification, key information extraction, and context-based question answering – into a single bilingual pipeline, as motivated in Chapter 1 and supported by the research gaps identified in Chapter 2. The design is further grounded in the characteristics of the DocSaarthi dataset established through the collection process in Chapter 3 and the preprocessing and exploratory analysis carried out in Chapter 4.

5.2 Proposed System Architecture

The proposed system architecture illustrates the major components of the DocSaarthi pipeline, from document acquisition to bilingual response generation.

Proposed NLP System Architecture for DocSaarthi

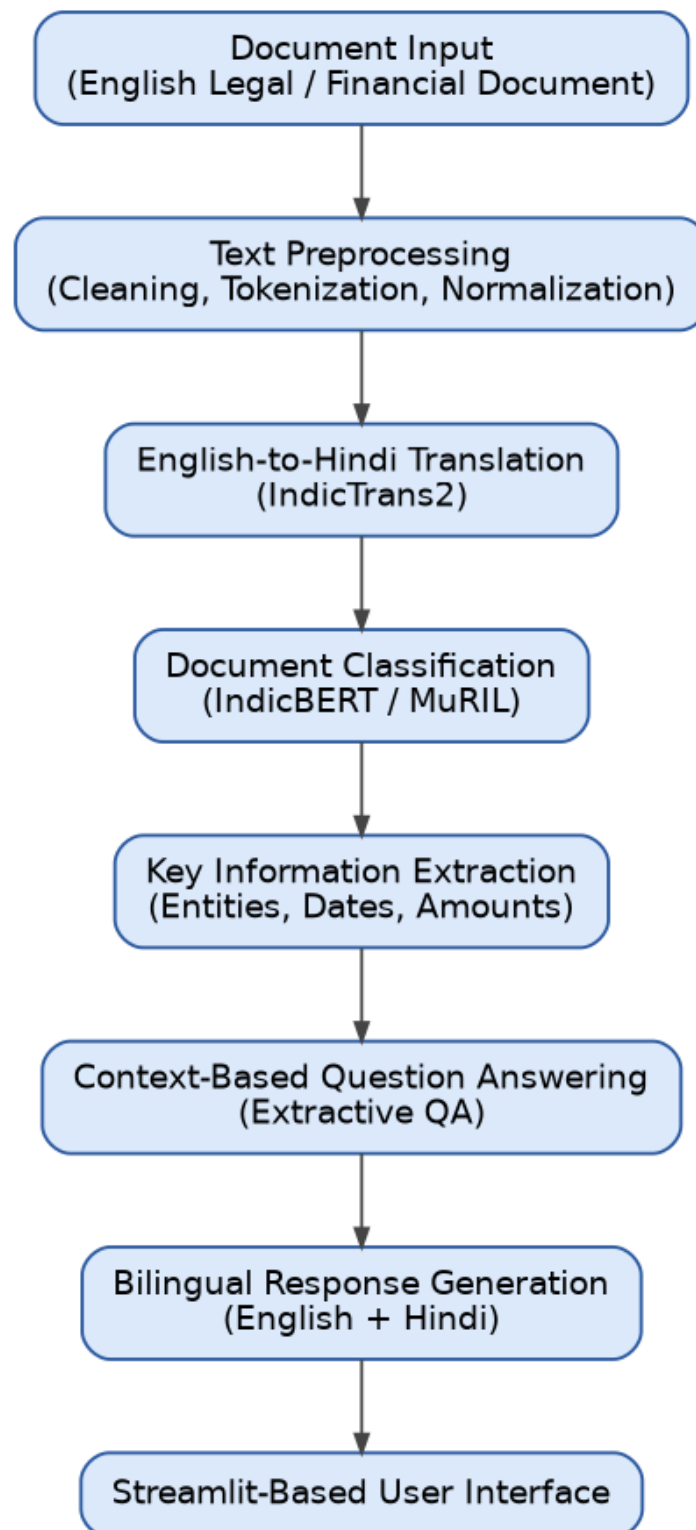


Figure 5.1: Proposed NLP System Architecture for DocSaarthi

The architecture consists of the following components:

1. **Document Input:** A user uploads or pastes an English legal or financial document (service agreement, loan contract, insurance policy, tax notice, court

proceeding excerpt, or similar) into the system.

2. **Text Preprocessing:** The input text undergoes cleaning, tokenization, and normalization, following the preprocessing design established in Chapter 4, so that it is in a suitable form for translation and downstream modelling.
3. **English-to-Hindi Translation:** The preprocessed document is translated from English into Hindi using a pre-trained multilingual translation model (IndicTrans2 [1]), producing a Hindi version of the document alongside the original English text.
4. **Document Classification:** The translated (or original) text is passed to a fine-tuned transformer classifier that assigns a `Document_Type` label (Legal or Financial) and a finer-grained `Intent` label (e.g. Agreement, Loan, Tax), consistent with the label scheme of the DocSaarthi dataset described in Chapter 3.
5. **Key Information Extraction:** The classified document is scanned for salient fields such as party names, monetary amounts, dates, and durations, using pattern-based rules combined with contextual embeddings from the classification model.
6. **Context-Based Question Answering:** A user may pose a natural-language question about the document; an extractive question-answering module locates the answer span within the document text, grounding every response strictly in the uploaded document.
7. **Bilingual Response Generation:** The classification result, extracted key information, and question-answering output are rendered to the user in both English and Hindi.
8. **Streamlit-Based User Interface:** All of the above stages are exposed to the user through a single interactive Streamlit interface, allowing document upload, translation and classification viewing, and natural-language querying without requiring the user to invoke separate tools.

5.3 Workflow Diagram

Figure 5.2 illustrates the complete DocSaarthi workflow from raw document input to the final bilingual output.



Figure 5.2: Proposed NLP Workflow for DocSaarthi

The workflow consists of the following stages:

1. **Dataset Collection:** Use of the 400-record, dual-labelled (Document_Type, Intent) Hindi legal/financial dataset described in Chapter 3 for classification and extraction experiments.
2. **Data Cleaning and Normalization:** Unicode and whitespace normalization, as applied to the dataset in Chapter 4.
3. **Tokenization:** Word-level tokenization for exploratory analysis and sub-word tokenization for transformer model input.
4. **Translation:** English input documents are translated into Hindi using Indic-Trans2 prior to, or in parallel with, classification.
5. **Feature Extraction:** Contextual sentence and token embeddings are obtained from the selected transformer encoder (Section 5.4.2).
6. **Model Training / Fine-Tuning:** The classification model is fine-tuned on the labelled DocSaarthi dataset; the question-answering module is configured for extractive answer-span prediction over document context.
7. **Prediction / Inference:** At inference time, an uploaded document is classified, key information is extracted, and any user question is answered using the document context.
8. **Bilingual Output Generation:** Classification labels, extracted fields, and question-answering responses are presented to the user in both English and Hindi.
9. **Performance Evaluation:** Each stage is evaluated independently using the metrics defined in Section 5.7.

5.4 Feature Representation Strategy

The effectiveness of an NLP model largely depends on how textual information is represented. Various candidate feature extraction techniques, spanning classical count-based methods and modern contextual embeddings, are considered before selecting the most appropriate representation for the bilingual, domain-specific text used in DocSaarthi.

5.4.1 Candidate Feature Representation Techniques

Technique	Description
Bag of Words (BoW)	Represents documents using word occurrence counts.
Count Vectorizer	Converts text into frequency vectors.
TF-IDF	Assigns weights based on word importance.
Word2Vec	Dense distributed word embeddings.
GloVe	Global word embedding representation.
FastText	Uses character-level information.
Doc2Vec	Generates document-level embeddings.
BERT	Context-aware transformer embeddings.
RoBERTa	Optimized transformer-based representation.
DistilBERT	Lightweight version of BERT.
IndicBERT	Transformer model for Indian languages.
MuRIL	Multilingual representation for Indian languages.
XLM-R	Cross-lingual multilingual transformer.

5.4.2 Selected Feature Representation

Contextual transformer-based embeddings are selected as the feature representation for DocSaarthi, in preference to classical count-based methods (BoW, TF-IDF, Count Vectorizer) and static embeddings (Word2Vec, GloVe, FastText).

- **Selected representation:** Contextual sub-word embeddings produced by an Indian-language-aware transformer encoder (IndicBERT or MuRIL [2]) for the Hindi classification and extraction stages, and the encoder representations internal to IndicTrans2 [1] for the translation stage.
- **Justification:** Static embeddings and count-based representations cannot capture the polysemy and clause-level context that distinguish, for example, a loan agreement clause from a tax clause containing similar vocabulary. Transformer-based contextual embeddings, further pre-trained on Indian-language corpora, are better suited to the code-mixed, template-driven Hindi text identified during the vocabulary analysis in Chapter 4 (Section 4.4).

- **Expected advantages:** Strong out-of-the-box performance on Hindi through Indic-specific pre-training; sub-word tokenization that gracefully handles the limited and repetitive 1,196-word vocabulary of the dataset without excessive out-of-vocabulary issues; transferability from general-domain pre-training to the legal/financial domain through fine-tuning on a relatively small labelled set.
- **Computational requirements:** Fine-tuning a base-sized transformer (110–180M parameters) on the 400-record dataset is feasible on a single consumer-grade GPU or a free-tier cloud GPU runtime, in line with the hardware constraints identified in Chapter 1.

5.5 Model Selection Strategy

Different machine learning and deep learning models are considered before selecting the final models for each pipeline stage.

5.5.1 Candidate Models

Model	Description
Naïve Bayes	Probabilistic classifier.
Logistic Regression	Linear classification model.
Support Vector Machine	Margin-based classifier.
Decision Tree	Tree-based supervised learning model.
Random Forest	Ensemble learning approach.
CNN	Captures local contextual features.
RNN	Sequential learning model.
LSTM	Handles long-term dependencies.
Bi-LSTM	Bidirectional sequential learning.
GRU	Lightweight recurrent model.
Transformer	Attention-based architecture.
BERT	Bidirectional transformer encoder [6].

Model	Description
IndicBERT	Transformer for Indian languages.
MuRIL	Multilingual Indian language model.
T5	Text-to-text transformer model.
mT5	Multilingual T5 model.
IndicTrans2	Indic-language neural machine translation model.

5.5.2 Selected Model

A stage-wise model selection is adopted, since DocSaarthi is a multi-task pipeline rather than a single classification problem:

- **Translation stage:** IndicTrans2 [1] is selected for English-to-Hindi translation, since it is an open-source model trained specifically for all 22 scheduled Indian languages, including Hindi, and was identified in the literature review (Chapter 2) as the leading available option for this sub-task.
- **Classification stage:** IndicBERT or MuRIL [2] is selected for the `Document_Type` (Legal/Financial) and `Intent` classification tasks, fine-tuned on the 400-record labelled dataset described in Chapter 3.
- **Key information extraction stage:** The same fine-tuned classification encoder is reused with a token-level (sequence-tagging) head over the extracted entity types (party names, dates, amounts, durations), avoiding the cost of training a separate large model from scratch.
- **Question answering stage:** An extractive question-answering formulation, using a transformer encoder with a span-prediction head over the document context, is selected in preference to a generative approach, so that every answer remains grounded in the source document text.
- **Selection criteria:** Availability of Hindi/Indic pre-training, open-source licensing, feasibility of fine-tuning within the hardware constraints of the micro project, and compatibility with the 400-record dataset size.
- **Advantages:** Reuse of pre-trained Indic language models substantially reduces the data and compute required compared to training from scratch; a shared encoder across classification and extraction reduces pipeline complexity.

- **Expected performance:** Given the dataset’s balanced `Document_Type` split and template-driven, low-vocabulary-diversity text (Chapter 4), high classification accuracy is expected; translation and question-answering performance is expected to be more sensitive to the fluency of the underlying pre-trained models on domain-specific legal/financial phrasing.
- **Possible limitations:** The 400-record dataset is small relative to typical transformer fine-tuning corpora, the four Financial intent sub-classes are comparatively under-represented (Chapter 4, Section 4.3), and none of the selected models are pre-trained specifically on Indian legal or financial text, which may limit their handling of domain-specific terminology.

5.6 System Requirements

5.6.1 Hardware Requirements

Table 5.3: Hardware Requirements

Processor	Intel Core i5 (10th generation or later) / equivalent, quad-core, 2.5 GHz or above
RAM	8 GB minimum; 16 GB recommended for transformer fine-tuning
Storage	At least 10 GB of free disk space for the dataset, pre-trained model checkpoints (IndicTrans2, IndicBERT/MuRIL), and supporting libraries
GPU (if required)	NVIDIA GPU with 6 GB or more VRAM recommended for fine-tuning; a free-tier cloud GPU runtime (e.g. Google Colab, Kaggle Notebooks) may be used as a substitute when local GPU hardware is unavailable
Internet Connectivity	Required for downloading pre-trained models (IndicTrans2, IndicBERT, MuRIL) and Python packages

5.6.2 Software Requirements

Table 5.4: Software Requirements

Operating System	Windows 10/11, Ubuntu 20.04 or later, or equivalent
Programming Language	Python 3.9 or later
IDE	Jupyter Notebook / VS Code / Google Colab
Libraries	pandas, NumPy, Matplotlib/Seaborn (EDA, per Chapter 4); NLTK, Indic NLP Library (preprocessing); HuggingFace Transformers, PyTorch (classification, extraction, question answering); IndicTrans2 toolkit (translation); Streamlit (user interface)
Version Control	Git / GitHub

5.7 Proposed Evaluation Strategy

5.7.1 Automatic Evaluation

Since DocSaarthi integrates four distinct NLP sub-tasks, each pipeline stage is evaluated independently using metrics appropriate to that sub-task.

Pipeline Stage	Evaluation Metrics
English-to-Hindi Translation	BLEU, chrF++
Document Classification (Legal/Financial, Intent)	Accuracy, Precision, Recall, F1-score
Key Information Extraction	Entity-level Precision, Recall, F1-score
Context-Based Question Answering	Exact Match (EM), F1-score

Given the moderate Intent class imbalance observed in Chapter 4 (Section 4.3), macro-averaged Precision, Recall, and F1-score are reported for the classification stage in addition to overall accuracy, so that performance on the less frequent Financial intent sub-classes is not obscured by the majority classes.

5.7.2 Human Evaluation

- **Number of evaluators:** Two to three evaluators (peers and/or faculty guide), sufficient for a semester-long micro project.
- **Evaluation criteria:** Fluency, correctness, relevance, adequacy, coherence, and script correctness of the Hindi output.
- **Rating scale:** A 5-point Likert scale (1 = poor to 5 = excellent) applied per criterion.
- **Evaluation procedure:** A random sample of translation outputs and question-answering responses is reviewed by evaluators without access to the automatic metric scores, to avoid anchoring bias.
- **Agreement method (optional):** Percentage agreement between evaluators is reported where more than one evaluator rates the same sample.

5.8 Proposed and Implemented Pipeline

The initial proposal considered transformer-based translation, classification, extraction, and question answering. The implemented Review 3 prototype adopts a lighter architecture: `pdfjs-dist` for page-level PDF extraction, Unicode-aware preprocessing, lexical Legal/Financial classification, pattern-based information extraction, chunk and token-based retrieval, a 400-record Hindi intent reference set, an offline English-Hindi translation memory, and source-grounded answer generation. OpenAI-based enhancement is optional and disabled by default.

This implementation preserves the project objective while reducing infrastructure cost and hallucination risk. The uploaded PDF remains the only primary source for document facts; the intent dataset and translation memory support routing and language presentation but never replace document evidence.

Suggested evaluation criteria specific to the bilingual nature of DocSaarthi:

- Fluency of the Hindi translation
- Correctness of the classification label with respect to document content
- Relevance of the extracted key information to the document
- Adequacy and groundedness of question-answering responses in the source document
- Coherence of the bilingual (English and Hindi) response presentation

- Readability for a non-expert user
- Script correctness (Devanagari rendering) of Hindi output
- Cultural and domain appropriateness of legal/financial terminology used

Chapter Summary

This chapter presented the proposed NLP pipeline for DocSaarthi, comprising a system architecture and workflow that carry an English legal or financial document through preprocessing, English-to-Hindi translation (IndicTrans2), document classification and key information extraction (IndicBERT/MuRIL), context-based question answering, and bilingual response generation, exposed to the user through a Streamlit-based interface. Contextual transformer-based embeddings were selected as the feature representation strategy, and a stage-wise model selection approach was adopted across the four pipeline sub-tasks. Hardware and software requirements were defined, and an evaluation strategy combining automatic metrics (BLEU/chrF++, Accuracy/F1, Entity-F1, EM/F1) with structured human evaluation was proposed for each stage. These design decisions provide the foundation for implementing and evaluating the proposed DocSaarthi pipeline in subsequent project stages.

Chapter 6

System Implementation

6.1 Introduction

This chapter describes the implementation of the proposed **DocSaarthi: AI Legal & Financial Document Assistant**. The system is implemented as a bilingual document-intelligence prototype that processes legal and financial PDF documents and provides structured information to the user.

The implementation includes PDF upload and validation, text extraction, language detection, legal/financial document classification, key information extraction, document summarization, bilingual retrieval, English–Hindi translation, and context-based question answering. The system also provides page-level evidence so that users can verify the generated information against the original document.

The implementation was developed as a local prototype using JavaScript and Node.js, with a 400-record Hindi intent dataset for question-intent routing. Translation is provided through the Sarvam translation API, while OpenAI-based answer generation is available as an optional enhancement. The implementation focuses on maintaining the uploaded document as the authoritative source of information.

6.2 Implementation Environment

The DocSaarthi prototype was implemented using a web-based client-server architecture. The backend handles document processing, text extraction, classification, retrieval, translation, and answer generation, while the frontend provides the user interface for document upload and interaction.

6.2.1 Hardware Environment

The following hardware configuration is suitable for implementing and testing the prototype.

Table 6.1: Hardware Environment

Component	Configuration
Processor	Intel Core i5 or equivalent processor
RAM	8 GB minimum; 16 GB recommended
Storage	At least 10 GB free storage
GPU	Not mandatory for the implemented prototype
Internet Connectivity	Required for external translation services and optional AI services

6.2.2 Software Environment

Table 6.2: Software Environment

Software / Technology	Purpose
Operating System	Windows 10/11 or equivalent
Programming Language	JavaScript
Runtime Environment	Node.js 20 or later
Frontend	HTML, CSS and JavaScript
Backend	Node.js based server application
PDF Processing	pdfjs-dist
Translation	Sarvam Translation API
Optional AI Service	OpenAI API
Dataset Format	CSV
Version Control	Git / GitHub

6.3 Dataset Implementation

The implemented system uses a labelled dataset containing **400 records** for intent classification. The dataset is used primarily for understanding and routing user

queries rather than as the evidence source for answering questions about uploaded documents.

The dataset contains Hindi-language intent examples representing different types of user queries related to legal and financial documents. The dataset is stored in CSV format and is loaded by the application during execution.

The document itself remains the primary source of truth. Therefore, information from the intent dataset is not directly used as evidence for answering document-related questions.

6.3.1 Dataset Loading

The dataset is loaded by the application from the configured CSV file. During initialization, the system checks whether the dataset is available and loads the records required for intent identification.

The implementation also provides a dataset inspection utility to check the number and distribution of records before execution.

6.3.2 Dataset Usage

The dataset is used for the following purposes:

- Identifying the probable intent of a user's question.
- Routing questions to the appropriate document-processing logic.
- Supporting bilingual question handling.
- Improving the interaction between the user and the document assistant.

The dataset is not treated as authoritative evidence for document answers. This separation helps prevent unrelated training examples from being incorrectly presented as information extracted from the uploaded document.

6.4 Preprocessing Implementation

The preprocessing stage converts the extracted PDF content into a clean and searchable textual representation. Since legal and financial documents contain important numbers, dates, percentages, names, and special symbols, preprocessing is performed carefully to avoid removing meaningful information.

The implemented preprocessing pipeline consists of PDF text extraction, Unicode normalization, line-break handling, duplicate removal, text cleaning, tokenization, and document chunking.

6.4.1 PDF Text Extraction

The uploaded PDF is processed page by page using the `pdfjs-dist` library. The extracted text is associated with its corresponding page number.

This page-level representation is important because the system uses the original page as evidence when displaying answers to the user.

6.4.2 Text Cleaning

The extracted text is cleaned before further processing. The implementation handles unnecessary line breaks, repeated whitespace, and common formatting inconsistencies while preserving important information such as:

- Names
- Dates
- Monetary values
- Percentages
- Legal clauses
- Financial terms
- Document identifiers

6.4.3 Unicode Normalization

Unicode normalization is applied to ensure that visually similar characters are represented consistently. This is particularly important for Hindi text because Devanagari characters may contain combining marks.

The preprocessing stage therefore preserves Devanagari characters instead of applying aggressive character removal.

6.4.4 Document Chunking

Long documents are divided into smaller text chunks before retrieval and translation. Chunking allows the system to search relevant sections efficiently and prevents excessively long text from being sent to external translation or AI services.

Each chunk retains its relationship with the original document page, allowing the final answer to provide page-level evidence.

6.5 Feature Representation Implementation

The implemented prototype uses a combination of lexical and structured textual representations rather than training a new transformer model from scratch.

The extracted document text is represented using normalized tokens and searchable text chunks. Token sets are used internally for matching user queries with relevant document sections.

6.5.1 Lexical Representation

Important terms from the document and user query are normalized and represented as token sets. These representations are used for retrieval and matching.

For example, a query containing terms such as "loan", "interest", "penalty", or "payment" can be matched against relevant portions of the uploaded financial document.

6.5.2 Bilingual Representation

The system supports English and Hindi interaction. A Hindi question can be processed against English document content by using the translation and retrieval layer.

Similarly, English queries can retrieve information from content that has been translated or represented in Hindi.

This provides cross-language retrieval without modifying the authoritative original document.

6.5.3 Structured Information Representation

In addition to textual chunks, the system maintains structured information extracted from the document, including:

- Dates
- Monetary values
- Percentages
- Important clauses
- Potential risk signals
- Document classification
- Page references

These representations help the system provide more useful and verifiable responses.

6.6 Model Implementation

The implementation uses a multi-stage approach rather than a single machine learning model.

6.6.1 Document Classification

The uploaded document is classified as either **Legal** or **Financial**. The current prototype uses document-content analysis and heuristic/lexical signals to determine the most probable category.

The system also provides a confidence value along with the predicted category.

6.6.2 Language Detection

The system detects whether the uploaded document contains English, Hindi, or mixed-language content. This information is used to determine the appropriate processing and translation flow.

6.6.3 Key Information Extraction

The implementation identifies important information from the document, including dates, monetary amounts, percentages, and potentially important clauses.

Regular expressions and domain-specific lexical patterns are used for extracting structured values.

6.6.4 Question Answering

The question-answering component follows a source-grounded approach. The user submits a question in English or Hindi. The system searches the indexed document chunks and retrieves the most relevant evidence.

The answer is generated only from the retrieved document content. When sufficient evidence cannot be found, the system is designed to return a “not found” response rather than inventing information.

6.6.5 Translation

English–Hindi translation is implemented using the Sarvam translation API. Translation is treated as a separate convenience layer and does not replace the original document.

The system supports:

- English-to-Hindi translation.
- Hindi-to-English translation where required.
- Hindi summaries.
- Bilingual question and answer interaction.

The implementation also includes text chunking and error handling for long translation requests.

6.6.6 Optional AI Answer Generation

OpenAI-based answer generation is implemented as an optional enhancement. It can generate more natural responses from retrieved document evidence.

The OpenAI component is disabled unless the required configuration and API key are explicitly provided. A local fallback mechanism is available when the external AI service is unavailable.

6.7 Training Configuration

The current DocSaarthi prototype does not perform large-scale transformer pretraining or fine-tuning. Instead, pretrained services and rule-based/lexical processing are combined to create a practical document-intelligence prototype.

The 400-record intent dataset is loaded during application execution and used for intent routing. The document-processing pipeline operates on the uploaded PDF at runtime.

Table 6.3: Training and Processing Configuration

Parameter	Configuration
Intent Dataset	400 labelled records
Dataset Format	CSV
Document Processing	Runtime processing of uploaded PDFs
Classification	Lexical/heuristic document analysis
Translation Model	Sarvam hosted translation service
Answer Generation	Local grounded logic with optional OpenAI enhancement
Retrieval	Document chunk and token-based matching
Model Fine-tuning	Not performed in the current prototype
GPU Requirement	Not mandatory

This configuration was selected because the project focuses on demonstrating an integrated multilingual document-intelligence pipeline within the available project resources.

6.8 Implementation Challenges and Solutions

During implementation, several challenges were identified due to the complexity of legal and financial documents and the requirement for bilingual interaction.

Table 6.4: Implementation Challenges and Solutions

Challenge	Implemented Solution
Long PDF documents	Documents are divided into smaller searchable chunks.
PDF line-break inconsistencies	Text normalization and line-break handling are applied.
Hindi Unicode handling	Unicode normalization preserves Devanagari characters and combining marks.
English–Hindi interaction	Cross-language retrieval and translation are implemented.
Large translation input	Translation requests are divided into manageable chunks.
Important numbers and dates	Monetary values, percentages, and dates are explicitly preserved and extracted.
Unsupported questions	The system returns a safe “not found” response when sufficient evidence is unavailable.
Hallucination risk	Answers are grounded in retrieved document evidence.
External API failure	Fallback mechanisms and error handling are implemented.
Document verification	Page-level citations are retained with retrieved evidence.

6.9 Implemented System Architecture

The implemented architecture follows a modular document-processing workflow. The major stages of the system are shown below.

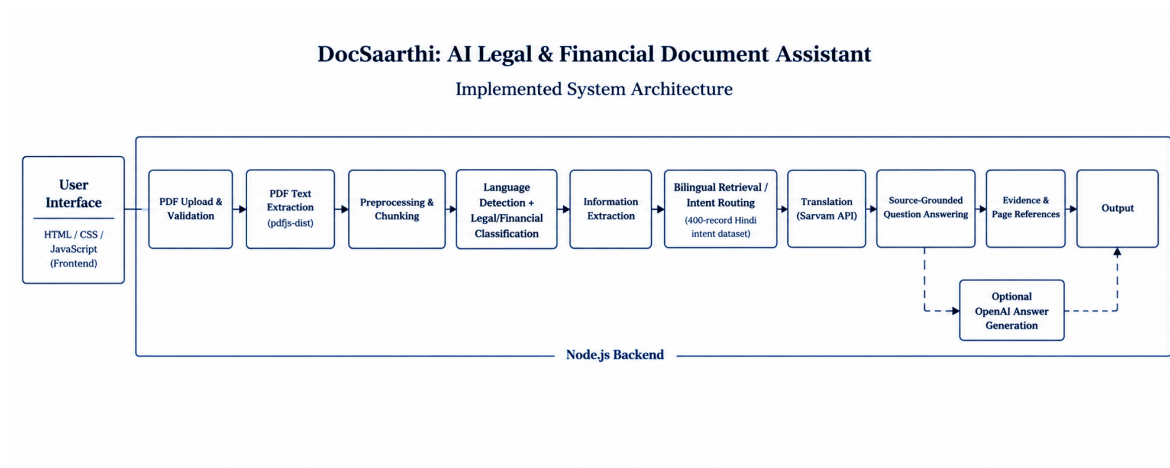


Figure 6.1: Implemented System Architecture of DocSaarthi

The implemented system consists of the following major components:

1. **User Interface:** Allows the user to upload documents, view results, and interact with the document assistant.
2. **PDF Upload and Validation:** Validates the uploaded PDF and prepares it for processing.
3. **PDF Text Extraction:** Extracts text from each page while preserving page information.
4. **Preprocessing Module:** Performs text normalization, cleaning, and chunking.
5. **Language Detection:** Identifies the language or language combination present in the document.
6. **Document Classification:** Determines whether the document is primarily Legal or Financial.
7. **Information Extraction:** Extracts dates, amounts, percentages, clauses, and potential risk signals.
8. **Bilingual Retrieval:** Matches user questions with relevant document chunks in English or Hindi.
9. **Translation Module:** Provides English–Hindi translation using the configured translation service.
10. **Question Answering Module:** Generates answers based on retrieved document evidence.
11. **Evidence and Citation Module:** Maintains page-level references so that users can verify the answer against the original document.

12. **Output Module:** Displays classification, extracted information, summaries, translations, and question-answering results to the user.

The implemented architecture follows a source-grounded design in which the original uploaded document remains the authoritative source. Translated content is maintained separately, and information from the intent dataset is not used as evidence for document answers.

Chapter Summary

This chapter presented the implementation of the proposed **DocSaarthi** system. The implementation environment, dataset handling, preprocessing pipeline, feature representation, document classification, information extraction, translation, and question-answering components were described. The chapter also discussed the processing configuration, implementation challenges and their solutions, and the final implemented system architecture.

The implemented prototype integrates PDF processing, bilingual interaction, document retrieval, structured information extraction, translation, and source-grounded question answering into a single workflow. The next chapter presents the experimental results and evaluation of the implemented system.

Chapter 7

Experimentation and Model Development

7.1 Introduction

This chapter presents the experimentation and model-development process carried out for the proposed **DocSaarthi: AI Legal & Financial Document Assistant**. The objective of the experimentation stage was to evaluate different processing strategies for document classification, information retrieval, bilingual interaction, translation, and source-grounded question answering.

The experiments were designed according to the implemented prototype described in the previous chapter. Since the current implementation does not perform large-scale transformer fine-tuning, the experiments focus on lexical and rule-based processing, document chunk matching, intent routing, translation, and optional AI-based answer generation.

The experimentation process was used to identify a practical configuration that provides reliable document-based responses while preserving important legal and financial information such as dates, monetary values, percentages, clauses, and page references.

7.2 Experimental Design

The experimental design was divided into multiple stages to evaluate the individual components of the DocSaarthi pipeline.

The major experimental stages were:

1. Preparation and inspection of the 400-record Hindi intent dataset.
2. Testing of PDF text extraction and page-level text preservation.

3. Evaluation of text preprocessing and document chunking.
4. Testing of lexical query-to-document matching.
5. Evaluation of Legal and Financial document classification.
6. Testing of English–Hindi translation.
7. Evaluation of source-grounded question answering.
8. Comparison of local answer generation with optional AI-assisted answer generation.
9. Evaluation of page-level evidence and unsupported-question handling.

The experimental workflow is summarized below.

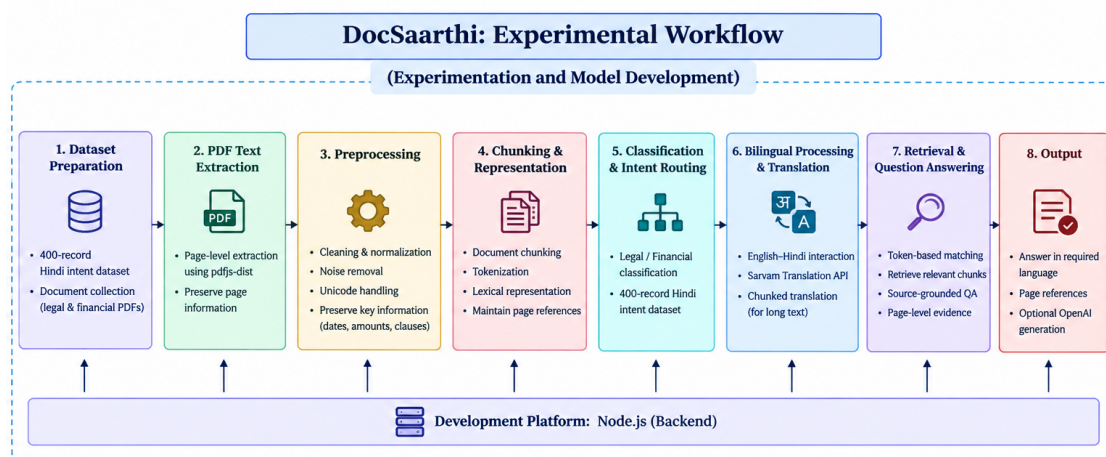


Figure 7.1: Experimental Workflow of DocSaarthi

Figure 7.1: Experimental Workflow of DocSaarthi

The same document set was used wherever possible so that different processing configurations could be compared under similar conditions.

7.3 Baseline Model

A baseline configuration was established to provide a reference for evaluating the proposed document-intelligence pipeline.

The baseline approach uses direct document text processing with basic lexical matching and limited preprocessing. User queries are matched against the extracted document text without applying the complete bilingual retrieval and evidence-handling workflow.

The baseline configuration consists of:

- Basic PDF text extraction.
- Basic text cleaning.
- Direct keyword-based matching.
- Simple document category identification.
- Direct response generation from matched text.

The baseline provides a simple reference implementation against which the improvements introduced by the proposed configuration can be observed.

Table 7.1: Baseline Configuration

Component	Baseline Configuration
PDF Processing	Basic page-level text extraction
Preprocessing	Basic cleaning and whitespace handling
Representation	Keyword/token representation
Retrieval	Direct lexical matching
Classification	Basic content-based classification
Translation	External translation service when required
Question Answering	Direct response from matched content
Evidence	Limited page-level reference handling

The baseline experiment was useful for identifying the limitations of simple keyword matching, especially for long documents and cross-language questions.

7.4 Proposed Model

The proposed configuration extends the baseline approach by introducing a modular source-grounded document-processing pipeline.

The proposed system combines preprocessing, document chunking, lexical retrieval, bilingual interaction, structured information extraction, translation, and evidence-based question answering.

The major components of the proposed configuration are:

1. PDF upload and validation.
2. Page-level PDF text extraction.
3. Unicode normalization and text cleaning.
4. Document chunking.
5. Legal/Financial document classification.
6. Structured information extraction.
7. Hindi intent routing using the 400-record dataset.
8. Bilingual query processing.
9. Token-based document retrieval.
10. English–Hindi translation using the Sarvam API.
11. Source-grounded question answering.
12. Page-level evidence presentation.
13. Optional OpenAI-based answer generation.

The proposed configuration maintains the original document as the authoritative source of information. The intent dataset is used for question routing and is not treated as evidence for document answers.

Table 7.2: Proposed Model Configuration

Component	Proposed Configuration
PDF Processing	Page-level extraction using pdfjs-dist
Preprocessing	Cleaning, Unicode normalization and chunking
Representation	Normalized tokens and document chunks
Classification	Lexical/heuristic Legal-Financial classification
Intent Routing	400-record Hindi intent dataset
Retrieval	Token and lexical document matching
Translation	Sarvam Translation API
Question Answering	Source-grounded document QA
Answer Enhancement	Optional OpenAI-based generation
Evidence	Page-level document references
Fallback	"Not found" response for insufficient evidence

7.5 Feature Representation Experiments

Feature representation experiments were conducted to determine how document and query text should be represented for effective matching.

Since the current prototype does not train a new transformer model, the experiments focused on lexical and structured representations.

7.5.1 Raw Text Representation

In the initial experiment, extracted document text was retained as plain text. This representation was useful for preserving the original content but was less effective for direct matching in long documents.

Long documents contained formatting inconsistencies, repeated whitespace, and line breaks that could reduce the effectiveness of simple matching.

7.5.2 Normalized Token Representation

The next experiment applied text normalization and tokenization. Important terms were converted into a consistent representation before matching.

This representation improved the ability to identify relevant document sections when the query and document used similar terminology.

7.5.3 Chunk-Based Representation

The document was divided into smaller searchable chunks while retaining the original page reference.

Chunk-based representation improved retrieval because the system could identify smaller portions of a document rather than processing the complete document for every question.

7.5.4 Structured Information Representation

Structured information such as dates, monetary values, percentages, clauses, and document classification was maintained separately from the general text representation.

This representation is particularly useful for legal and financial questions where numerical and contractual information must be preserved accurately.

Table 7.3: Feature Representation Experiments

Representation	Purpose	Observation
Raw Text	Preserve extracted document content	Simple but less suitable for long-document matching
Normalized Tokens	Improve lexical matching	More consistent query matching
Document Chunks	Retrieve relevant sections	Improved focused retrieval and evidence tracking
Structured Values	Preserve dates and numerical information	Useful for legal and financial queries

7.6 Model Comparison

The baseline and proposed configurations were compared using functional and qualitative criteria relevant to the DocSaarthi application.

The comparison focused on retrieval quality, bilingual support, document verification, handling of long documents, and resistance to unsupported answers.

Table 7.4: Baseline and Proposed Model Comparison

Parameter	Baseline	Proposed
Text Preprocessing	Basic	Normalization and cleaning
Document Chunking	Limited	Implemented
Query Matching	Direct lexical matching	Chunk and token-based matching
Bilingual Interaction	Limited	English–Hindi support
Intent Routing	Not emphasized	400-record Hindi intent dataset
Information Extraction	Basic	Structured extraction
Translation	External translation when required	Integrated translation layer
Evidence Tracking	Limited	Page-level evidence
Unsupported Questions	May produce weak matches	“Not found” fallback
Answer Generation	Direct response	Grounded response with optional AI enhancement

The proposed configuration provides a more complete workflow for document understanding because retrieval, translation, evidence tracking, and answer generation are integrated into the same pipeline.

7.7 Hyperparameter Experimentation

The current prototype does not perform neural network training or transformer fine-tuning. Therefore, conventional training hyperparameters such as learning rate, batch size, and number of epochs are not applicable to the implemented system.

Instead, experimentation was performed on processing parameters that affect document retrieval and response generation.

The important configurable parameters include:

- Document chunk size.
- Amount of text overlap between chunks.
- Number of retrieved document chunks.
- Lexical matching threshold.
- Maximum translation text length.
- Query normalization rules.
- Answer-generation configuration.

These parameters were adjusted to achieve a balance between retrieval coverage, response relevance, processing time, and API limitations.

Table 7.5: Processing Parameter Experimentation

Parameter	Effect of Increasing Value	Experimental Consideration
Chunk Size	More context per retrieval unit	Very large chunks may reduce retrieval precision
Chunk Overlap	Better continuity between sections	Excessive overlap increases repeated content
Retrieved Chunks	More contextual information	Too many chunks may introduce irrelevant text
Matching Threshold	Stricter relevance filtering	Excessive threshold may miss valid evidence
Translation Chunk Length	Larger translation context	Very large inputs may exceed service limits

The final processing configuration was selected based on practical testing with the implemented document-processing workflow.

7.8 Experimental Observations

The experiments produced several important observations.

7.8.1 Observation 1: Importance of Preprocessing

Raw PDF text cannot always be used directly for reliable retrieval. Line breaks, repeated spaces, and formatting inconsistencies can affect lexical matching. Text normalization therefore improves the consistency of the document representation.

7.8.2 Observation 2: Importance of Chunking

Chunking improves retrieval for long legal and financial documents. Instead of searching the entire document as a single unit, the system can identify smaller sections that are more closely related to the user's question.

7.8.3 Observation 3: Importance of Page References

Maintaining page information during extraction and chunking is important for document verification. Users can trace the generated response back to the relevant section of the original document.

7.8.4 Observation 4: Bilingual Interaction

Translation enables users to interact with English documents using Hindi questions and responses. However, translation is maintained as a separate processing layer so that the original English document remains unchanged.

7.8.5 Observation 5: Source Grounding

Restricting answers to retrieved document evidence reduces the risk of providing information that is not present in the uploaded document. When adequate evidence cannot be retrieved, the system can return a “not found” response.

7.8.6 Observation 6: External Service Dependency

The translation and optional AI answer-generation components depend on external services. Therefore, error handling and fallback mechanisms are required to maintain system usability when external services are unavailable.

7.8.7 Observation 7: Intent Dataset Usage

The 400-record Hindi dataset is useful for identifying and routing user query intents. However, it is not used as the authoritative knowledge source for document answers.

7.9 Selected Model and Configuration

Based on the experimentation, the proposed source-grounded configuration was selected for the DocSaarthi prototype.

The selected configuration combines lexical document retrieval, document chunking, structured information extraction, bilingual translation, intent routing, and page-level evidence.

Table 7.6: Selected Model and Configuration

Component	Selected Configuration
Programming Language	JavaScript
Runtime	Node.js 20 or later
PDF Processing	pdfjs-dist
Text Representation	Normalized tokens and document chunks
Document Classification	Lexical/heuristic analysis
Intent Dataset	400 labelled Hindi records
Retrieval	Token and lexical matching
Translation	Sarvam Translation API
Question Answering	Source-grounded document QA
AI Enhancement	Optional OpenAI answer generation
Evidence	Page-level references
Fine-tuning	Not performed

The selected configuration provides a practical balance between implementation complexity, processing requirements, bilingual functionality, and document-grounded response generation.

7.10 Stage 3 Readiness

The experimentation stage prepares the DocSaarthi prototype for the next stage of development and evaluation.

The following components are ready for further testing:

- PDF upload and validation.
- Page-level PDF text extraction.
- Text preprocessing and normalization.
- Document chunking.
- Legal/Financial document classification.

- Structured information extraction.
- Hindi intent dataset integration.
- English–Hindi translation.
- Bilingual document retrieval.
- Source-grounded question answering.
- Page-level evidence presentation.
- Optional AI-based answer generation.

The next stage can focus on systematic evaluation of the implemented system using representative legal and financial documents. Evaluation can include retrieval relevance, classification correctness, translation quality, answer correctness, evidence coverage, response time, and handling of unsupported questions.

The system can also be extended in future experiments by introducing trained multilingual models such as IndicBERT, MuRIL, or IndicTrans2 when sufficient computational resources and labelled training data are available. Such models are not considered part of the current implemented prototype.

Chapter Summary

This chapter presented the experimentation and model-development process for the **DocSaarthi** system. The experimental design, baseline configuration, proposed configuration, feature representation experiments, model comparison, processing parameter experimentation, and experimental observations were discussed.

The experiments showed the importance of preprocessing, document chunking, lexical retrieval, bilingual interaction, structured information extraction, and page-level evidence for building a reliable legal and financial document assistant.

The source-grounded proposed configuration was selected as the final prototype configuration. The system is now prepared for systematic evaluation and further development in the next stage of the project.

Chapter 8

Individual Technical Contribution

8.1 Introduction

This chapter describes the individual technical contributions made by the two team members during the development of **DocSaarthi: AI Legal & Financial Document Assistant**.

The project integrates Natural Language Processing techniques for English–Hindi translation, legal and financial document classification, key information extraction, and bilingual question answering. The implemented system also includes PDF processing, document preprocessing, document chunking, retrieval, source-grounded question answering, and page-level evidence verification.

The two team members, **Padmaja Shinde** and **Akshata Naik**, contributed to different technical and development activities throughout the project. Padmaja primarily contributed to the core document-processing and NLP pipeline, while Akshata contributed to bilingual interaction, testing, validation, translation verification, and system-level integration.

Both members participated in debugging, testing, validation, and technical documentation.

8.2 Individual Contribution Matrix

Table 8.1 presents the major technical contributions of Padmaja Shinde and Akshata Naik.

Table 8.1: Individual Contribution Matrix

Technical Area	Padmaja Shinde	Akshata Naik
Requirement Analysis	Studied the project requirements and overall DocSaarthi workflow.	Studied user interaction requirements and bilingual document-assistance requirements.
PDF Processing	Worked on PDF text extraction and page-level document processing.	Tested PDF processing and verified extracted content and page references.
Text Preprocessing	Worked on text cleaning, normalization, and preparation of document content.	Validated preprocessing results and checked preservation of important document information.
Document Chunking	Worked on document chunking for efficient retrieval of relevant content.	Tested chunk retrieval and relevance of retrieved document sections.
Intent Dataset	Worked with the 400-record Hindi intent dataset for query routing.	Reviewed and tested Hindi intent examples and query-routing behaviour.
Bilingual Processing	Worked on English–Hindi bilingual processing and interaction flow.	Tested English, Hindi, and bilingual queries through the implemented system.
Translation	Integrated and tested the Sarvam Translation API.	Validated English–Hindi translation results and checked important terms and values.
Information Extraction	Worked on extraction of dates, monetary values, percentages, important clauses, and document information.	Verified extracted information and checked correctness of displayed values.
Question Answering	Worked on testing source-grounded question answering.	Validated generated answers against the corresponding document evidence.
Testing and Debugging	Performed document processing, retrieval, and integration testing.	Performed bilingual, translation, output, and system validation testing.
Documentation	Contributed to technical documentation and report preparation.	Contributed to testing records, documentation, and presentation preparation.

8.3 Individual Technical Tasks

The following subsections describe the technical activities performed by each team member during the development of DocSaarthi.

8.3.1 Padmaja Shinde

Padmaja Shinde contributed primarily to the core document-processing and NLP pipeline of the proposed system.

The major technical tasks performed were:

- Analysed the requirements and overall workflow of the DocSaarthi system.
- Worked on PDF text extraction and page-level document processing.
- Worked on text preprocessing, cleaning, and normalization.
- Worked on document chunking to divide lengthy documents into manageable sections for retrieval.
- Worked with the 400-record Hindi intent dataset used for query routing.
- Contributed to English–Hindi bilingual processing.
- Integrated and tested the Sarvam Translation API.
- Worked on extraction of important document information, including:
 - Dates
 - Monetary values
 - Percentages
 - Important clauses
 - Other relevant document information
- Worked on testing source-grounded question answering.
- Participated in debugging and integration testing.
- Contributed to technical documentation and project report preparation.

These contributions correspond to the technical contribution activities identified for Padmaja in the project material.

8.3.2 Akshata Naik

Akshata Naik contributed primarily to bilingual interaction, validation, testing, and system-level verification of the DocSaarthi prototype.

The major technical tasks performed were:

- Studied the user requirements and interaction flow of the legal and financial document assistant.
- Participated in testing PDF upload and document processing functionality.
- Verified extracted document text and page-level references.
- Tested preprocessing and document chunking results.
- Tested query routing using the Hindi intent dataset.
- Tested English, Hindi, and bilingual user queries.
- Participated in testing the English–Hindi bilingual processing workflow.
- Tested the Sarvam Translation API and verified the quality of translated output.
- Checked whether important information such as dates, monetary values, percentages, names, and document identifiers remained correctly represented after translation.
- Tested document classification results for Legal and Financial documents.
- Tested retrieval of relevant document sections for user queries.
- Validated source-grounded question-answering responses against the uploaded document.
- Verified page-level evidence associated with generated answers.
- Tested unsupported questions and verified the safe “Not Found” response.
- Assisted in identifying and reporting implementation issues during system testing.
- Participated in debugging and integration testing.
- Contributed to technical documentation and presentation preparation.

8.4 Testing and Debugging Contribution

Testing and debugging were carried out collaboratively by both team members to ensure that the implemented system performed correctly across document processing, translation, retrieval, and question-answering tasks.

8.4.1 Padmaja Shinde

Padmaja contributed to testing and debugging of the core NLP and document-processing components.

The testing activities included:

- PDF text extraction testing.
- Text preprocessing and normalization testing.
- Document chunking testing.
- Document classification testing.
- Information extraction testing.
- Retrieval testing.
- Translation API integration testing.
- Source-grounded question-answering testing.
- Backend integration testing.

Padmaja also participated in identifying processing errors and verifying corrections after debugging.

8.4.2 Akshata Naik

Akshata contributed mainly to system-level and user-oriented testing.

The testing activities included:

- Testing English and Hindi queries.
- Testing bilingual interaction.
- Testing query routing and intent handling.
- Testing translation output.

- Verifying important values after translation.
- Testing retrieval relevance.
- Checking source-grounded answers against document content.
- Verifying page-level evidence.
- Testing unsupported questions.
- Identifying unexpected outputs during system execution.

These testing activities support the project requirement that the system provide bilingual responses and evidence that can be verified against the original document.

8.4.3 Joint Testing and Debugging

Both members jointly participated in end-to-end system testing.

The joint testing process included:

1. Uploading legal and financial PDF documents.
2. Checking successful PDF text extraction.
3. Verifying preprocessing and chunking.
4. Testing Legal and Financial classification.
5. Checking extraction of dates, monetary values, percentages, and important clauses.
6. Testing English and Hindi queries.
7. Testing translation.
8. Testing retrieval of relevant document chunks.
9. Verifying source-grounded answers.
10. Checking page-level evidence.
11. Testing unsupported questions.
12. Re-testing the system after debugging.

8.5 Problems Encountered and Solutions

Several technical challenges were encountered during the implementation of Doc-Saarthi. The major problems and the corresponding solutions are presented in Table 8.2.

Table 8.2: Problems Encountered and Implemented Solutions

Problem	Solution
Long PDF documents	Documents were divided into smaller chunks to make retrieval and processing more manageable.
PDF text formatting issues	Text cleaning, normalization, and line-break handling were applied during preprocessing.
Hindi Unicode handling	Unicode normalization was used while preserving Devanagari characters.
Bilingual interaction	English–Hindi translation and bilingual retrieval were incorporated into the processing workflow.
Long translation requests	Large text inputs were divided into smaller translation chunks.
Important numerical information	Dates, monetary values, percentages, and identifiers were preserved during processing.
Irrelevant retrieval	Normalized lexical and token-based matching was used to retrieve relevant document sections.
Unsupported questions	The system provides a safe “Not Found” response when sufficient evidence is unavailable.
Hallucination risk	Question answering is grounded in retrieved document evidence.
External API dependency	Error handling and fallback mechanisms were incorporated for external services.
Document verification	Page-level references were retained so that users could verify the generated information.

8.6 Skill Acquisition and Technology Adoption

The development of DocSaarthi provided both team members with practical experience in Natural Language Processing, document intelligence, bilingual processing, API integration, testing, and debugging.

8.6.1 Padmaja Shinde

The major technical skills acquired and strengthened by Padmaja include:

- PDF document processing.
- Page-level text extraction.
- Text preprocessing and normalization.
- Document chunking.
- Lexical document retrieval.
- Structured information extraction.
- Bilingual English–Hindi processing.
- Translation API integration.
- Source-grounded question answering.
- Backend integration and debugging.
- Technical documentation and report preparation.

8.6.2 Akshata Naik

The major technical skills acquired and strengthened by Akshata include:

- Understanding of NLP-based document processing.
- Working with a Hindi intent dataset.
- Bilingual English–Hindi interaction.
- Translation API testing.
- Document classification validation.
- Retrieval and question-answering validation.

- Verification of extracted legal and financial information.
- Functional testing and debugging.
- System integration testing.
- Technical documentation and presentation preparation.

8.6.3 Technology Adoption

The major technologies used during the implementation of DocSaarthi are summarized below.

Table 8.3: Technology Adoption

Technology	Application
HTML, CSS and JavaScript	Frontend interface and user interaction.
Node.js	Backend runtime and application processing.
pdfjs-dist	PDF text extraction and page-level processing.
400-record Hindi Dataset	Intent identification and query routing.
Sarvam Translation API	English–Hindi and Hindi–English translation.
Lexical / Token Matching	Document retrieval and query matching.
OpenAI API	Optional AI-assisted answer generation.
Git / GitHub	Version control and project development.

The project material identifies Node.js, HTML/CSS/JavaScript, pdfjs-dist, the 400 Hindi intent records, Sarvam Translation API, lexical/token-based retrieval, and optional OpenAI integration as the main implementation technologies.

8.7 Individual Contribution Evidence

The individual contribution of both members can be demonstrated through the technical implementation, testing activities, system outputs, and project documentation.

8.7.1 Evidence for Padmaja Shinde

The following activities provide evidence of Padmaja's technical contribution:

- Requirement analysis and understanding of the DocSaarthi workflow.
- PDF text extraction and page-level processing.
- Text preprocessing and normalization.
- Document chunking.
- Integration of the 400-record Hindi intent dataset.
- English–Hindi bilingual processing.
- Sarvam Translation API integration and testing.
- Extraction of dates, monetary values, percentages, and important clauses.
- Testing of source-grounded question answering.
- Debugging and integration testing.
- Technical documentation and report preparation.

These activities are explicitly listed in the project's individual contribution material.

8.7.2 Evidence for Akshata Naik

The following activities provide evidence of Akshata's contribution to the development and validation of the system:

- Testing of document upload and PDF processing.
- Validation of extracted document content.
- Testing of preprocessing and document chunking.
- Testing of the Hindi intent dataset and query routing.
- Testing English, Hindi, and bilingual queries.
- Translation API testing and output verification.
- Validation of extracted dates, monetary values, percentages, and important clauses.
- Testing of Legal and Financial document classification.

- Retrieval relevance testing.
- Source-grounded question-answering validation.
- Page-level evidence verification.
- Unsupported-question testing.
- Debugging and integration testing.
- Technical documentation and presentation support.

8.7.3 Project Artifacts as Evidence

The following project artifacts can be used to demonstrate the technical contributions of both members:

- Source-code files.
- Git/GitHub repository and commit history.
- Hindi intent dataset.
- PDF processing implementation.
- Translation API integration.
- Testing outputs.
- Screenshots of the implemented system.
- Classification and information extraction results.
- Question-answering results.
- Technical documentation.
- Project presentation and demonstration.

The project presentation also records the team's implementation technologies, including Node.js, pdfjs-dist, the 400-record Hindi dataset, Sarvam Translation API, lexical/token-based retrieval, and optional OpenAI integration.

Chapter Summary

This chapter presented the individual technical contributions of **Padmaja Shinde** and **Akshata Naik** in the development of DocSaarthi.

Padmaja Shinde primarily contributed to the core document-processing and NLP pipeline, including PDF text extraction, preprocessing, document chunking, intent dataset integration, bilingual processing, translation integration, information extraction, source-grounded question answering, debugging, and technical documentation.

Akshata Naik primarily contributed to system validation, bilingual interaction testing, translation verification, intent and query routing validation, document classification testing, retrieval verification, question-answering validation, debugging, and documentation.

Both members contributed to the integration, testing, debugging, and validation of the complete DocSaarthi system.

The contribution of Padmaja is directly documented in the project material, including requirement analysis, PDF processing, preprocessing, document chunking, intent dataset integration, bilingual processing, Sarvam integration, information extraction, question-answering testing, debugging, and documentation.

The combined contributions supported the implementation and validation of a bilingual, source-grounded legal and financial document assistant. The project architecture integrates document processing, classification, information extraction, bilingual retrieval, translation, question answering, and page-level evidence into a single workflow.

Chapter 9

Model Evaluation and Validation

9.1 Introduction

This chapter evaluates the implemented DocSaarthi prototype using repeatable automated tests and a labelled bilingual PDF holdout. The purpose is to verify domain classification, language detection, cross-language retrieval, translation fidelity, source grounding, and safe abstention.

9.2 Evaluation Methodology

Three complementary checks were used: 18 Node.js regression tests, an eight-document labelled holdout containing four Legal and four Financial PDFs in English and Hindi, and manual inspection of representative answers and citations. The holdout is synthetic and deliberately small; therefore, its scores demonstrate prototype correctness on the controlled benchmark and are not presented as production accuracy.

9.3 Evaluation Metrics

Classification is reported using accuracy, per-class precision, recall, F1-score, macro-F1, and a confusion matrix. Language detection is reported using accuracy. Retrieval and answer generation are validated through expected evidence matching, page-number preservation, value preservation, and abstention assertions.

9.4 Quantitative Evaluation

All 18 automated tests passed. The labelled holdout produced eight correct document-domain predictions and eight correct language predictions.

Table 9.1: Review 3 Quantitative Results

Measure	Result
Automated regression tests	18/18 passed
Document classification accuracy	100% (8/8)
Macro-averaged F1-score	1.00
Legal precision / recall / F1	1.00 / 1.00 / 1.00
Financial precision / recall / F1	1.00 / 1.00 / 1.00
Language detection accuracy	100% (8/8)

9.5 Model Performance

The rule-based classifier correctly separated all four Legal and four Financial samples. Displayed confidence values ranged from 0.82 to 0.84. These confidence values are bounded interface indicators rather than calibrated probabilities.

9.6 Comparative Evaluation

Compared with the earlier direct keyword baseline, the implemented pipeline adds page-linked chunks, bilingual query support, value-preserving translation checks, explicit grounding, and an abstention path. The comparison is functional because a sufficiently large real-world benchmark for statistically significant model comparison is not yet available.

9.7 Class-wise / Category-wise Evaluation

Table 9.2: Confusion Matrix for the Eight-Document Holdout

Actual Class	Predicted Legal	Predicted Financial
Legal	4	0
Financial	0	4

9.8 Human Evaluation (where applicable)

The two team members manually verified representative bilingual answers against the corresponding source excerpt and page number during development. A formal blinded study with external evaluators and Likert-scale ratings has not yet been completed; it is retained as future work to avoid overstating evidence.

9.9 Validation Results

Validation confirmed English-to-Hindi and Hindi-to-English retrieval, preservation of rupee amounts, percentages, dates, and names, safe rejection of Hinglish output, handling of PDF line breaks, preservation of Devanagari combining marks, and abstention for unsupported questions such as a missing PAN number.

Chapter Summary

The controlled Review 3 benchmark and regression suite validate the implemented prototype. The results are encouraging, but broader real-document and human evaluation is required before deployment claims can be made.

Chapter 10

Error Analysis and Model Improvement

10.1 Introduction

Error analysis was used to convert observed risks into regression tests and safeguards. Particular attention was given to unsupported answers, bilingual retrieval failures, translation corruption, PDF formatting artefacts, and misclassification.

10.2 Error Analysis Methodology

Each failure mode was reproduced with a minimal document and query. The expected source excerpt, output language, preserved values, page reference, or abstention behavior was then encoded as an automated assertion. A correction was accepted only after the original case and the full suite passed.

10.3 Error Categories

The principal categories were language and answer ordering, Legal/Financial classification, cross-language retrieval and grounding, translation fidelity, PDF extraction and processing, and Unicode robustness.

10.4 Representative Error Cases

Case	Risk	Implemented Response
------	------	----------------------

Unsupported PAN question	Fabricated identifier	Return a grounded “not found” answer with no citation
Hindi query on English PDF	No relevant cross-language match	Expand bilingual intent and terminology signals before retrieval
PDF line break inside sentence	Evidence split across chunks	Repair line breaks before sentence and chunk processing
Hinglish translation	Reduced readability and altered meaning	Reject invalid output using script and preservation checks
Amounts, dates, or percentages changed	Financial meaning corrupted	Validate protected values against the source
External translation failure	Upload or analysis blocked	Defer translation and retain the original source result

10.5 Error Distribution

The following table describes regression-test coverage, not production error frequency. Each of the 18 tests is assigned to its primary risk category.

Table 10.2: Distribution of Automated Validation Coverage

Primary Category	Tests	Share
Language and answer ordering	2	11.1%
Classification and confidence	2	11.1%
Retrieval, grounding, and abstention	5	27.8%
Translation fidelity and value preservation	6	33.3%
PDF extraction and processing	2	11.1%
Unicode robustness	1	5.6%
Total	18	100%

10.6 Analysis of Error Causes

The dominant causes are vocabulary mismatch between languages, unreliable layout in extracted PDFs, preservation-sensitive financial values, insufficient evidence, and dependency failure. These causes are architectural rather than a single trainable-model problem, so they require layered validation and fallback behavior.

10.7 Model Improvement

Improvements include Unicode normalization, line-break repair, page-linked chunking, bilingual retrieval signals, protected-value validation, on-demand translation, source-only answer grounding, and explicit abstention. Classification confidence is constrained to a narrow range to avoid presenting an uncalibrated score as certainty.

10.8 Before and After Improvement

Table 10.3: Before and After Improvement

Before	After
Weak match could be returned for missing facts	Unsupported questions abstain and expose no false source
Translation coupled to document processing	Upload remains usable even if translation fails
Line breaks could fragment evidence	Sentence continuity is restored before chunking
Mixed-language output could pass through	Hindi output is checked for natural script usage
Critical values could drift in translation	Names, dates, amounts, and percentages are verified

10.9 Discussion of Improvement

The final suite passing 18/18 confirms that the selected safeguards work for the encoded cases. It does not prove completeness: OCR errors, adversarial documents, unfamiliar terminology, and large-scale latency remain open evaluation areas.

Chapter Summary

The improvement process prioritized trustworthiness and traceability. Every material safeguard is connected to a repeatable test, while unresolved limitations are stated explicitly.

Chapter 11

Final System and Results

11.1 Introduction

The final Review 3 prototype is a bilingual legal and financial document assistant that accepts text-based PDFs, extracts and analyses their content, and answers questions using traceable source evidence.

11.2 Final System Architecture

The browser interface communicates with a Node.js and Express backend. The back-end coordinates PDF extraction, language detection, classification, information extraction, chunking, retrieval, translation, and grounded answer generation. In-memory storage is used in the prototype.

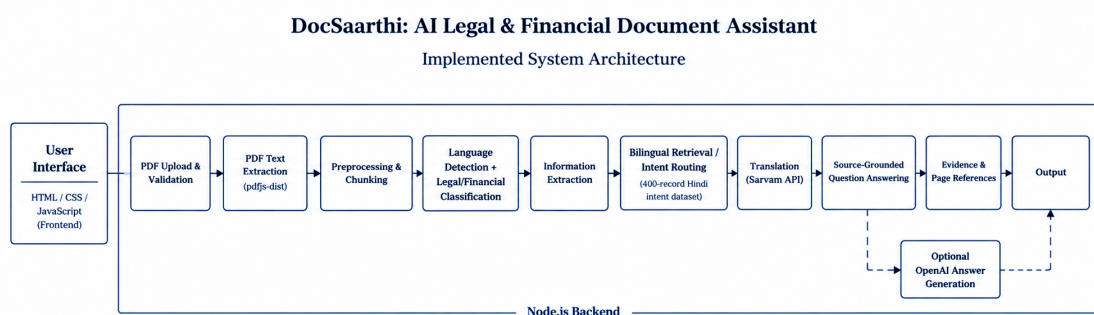


Figure 11.1: Final Implemented System Architecture

11.3 Final End-to-End Workflow

The user uploads a PDF; the server validates the file, extracts page-level text, detects language and document type, extracts important values, and builds searchable chunks. A user question is normalized and routed, relevant chunks are retrieved, an answer is formed only from those chunks, and the response includes page-linked evidence. Translation is performed on demand while the original PDF remains authoritative.

11.4 Final System Implementation

The implementation uses HTML, CSS, and JavaScript on the frontend; Node.js and Express on the backend; `pdfjs-dist` for extraction; local rule and lexical services for analysis and retrieval; and a 400-record Hindi dataset for intent signals. An offline English-Hindi parallel dataset supports known translations. OpenAI use is optional and controlled through configuration.

11.5 Final Results

The final build passes all 18 automated tests. On the eight-document labelled hold-out, the system correctly classified all documents and detected both English and Hindi correctly. Source citations, page numbers, abstention, and critical-value preservation also passed their regression checks.

11.6 Sample Outputs

Input Scenario	Expected System Behavior	Verified Output Characteristic
Hindi legal agreement with English termination query	Retrieve Hindi termination clause	Evidence contains the thirty-day clause and page 7 reference
English financial document with Hindi EMI query	Retrieve English repayment clause and answer in Hindi	Answer preserves the monthly EMI of ₹20,000 and cites the source excerpt

Question asking for absent PAN number	Do not infer an identifier	"Information not found" response, no source citation, grounded flag false
Financial translation request	Preserve protected values	Name, ₹5,00,000, 9.50%, and 05/08/2029 remain unchanged

11.7 Results Discussion

The results show that a lightweight pipeline can provide useful, traceable assistance for controlled legal and financial samples. The strongest outcome is not the small-sample accuracy alone, but the combination of cross-language retrieval, source citations, value preservation, and safe abstention.

11.8 Achievement of Project Objectives

The prototype satisfies the principal objectives: PDF processing, Legal/Financial classification, bilingual interaction, structured information extraction, question answering from document evidence, page-level verification, and an accessible web interface. It also separates supporting datasets from authoritative document evidence.

11.9 Limitations of the Final System

The system does not yet support scanned PDFs without OCR, persistent multi-user storage, authentication, calibrated confidence, exhaustive legal or financial terminology, or production-scale evaluation. The translation memory can translate only supported content, and the in-memory repository loses documents when the server restarts.

Chapter Summary

DocSaarthi delivers an operational end-to-end prototype with verified bilingual and grounding safeguards. The final results support academic demonstration while leaving clear work for production readiness.

Chapter 12

SDG, Sustainability, Safety and Societal Impact

12.1 Introduction

DocSaarthi is intended to make complex legal and financial documents easier to explore without replacing qualified professional advice. This chapter connects the prototype to sustainable development, safety, ethics, and responsible use.

12.2 SDG Alignment

The project primarily supports United Nations Sustainable Development Goal 16, especially access to information and more inclusive institutions, by presenting document-grounded information in English and Hindi. It also supports SDG 10 by reducing language-based barriers and SDG 4 by helping users learn the terminology and structure of formal documents.

12.3 Sustainability Considerations

The lightweight rule-based and lexical pipeline runs on modest hardware and avoids mandatory large-model inference for every request. On-demand translation prevents unnecessary work. Digital document analysis can reduce repeated printing, although server operation, storage, and optional external AI services still consume energy and should be monitored in future deployments.

12.4 Safety and Ethical Considerations

Legal and financial content is sensitive and high impact. The system therefore keeps the uploaded PDF as the primary source, displays evidence, preserves important values, abstains when evidence is insufficient, and describes its output as assistance rather than professional advice. Synthetic evaluation data reduces privacy risk during testing.

12.5 Societal Impact

Potential benefits include improved access for Hindi-speaking users, faster identification of clauses and values, and better preparation before seeking professional help. Potential harms include over-reliance, incorrect interpretation, exposure of confidential documents, and unequal performance across document styles or languages.

12.6 Responsible Use and Limitations

Users must verify answers against the displayed source page and consult a lawyer, accountant, bank, or other qualified professional for consequential decisions. A production system requires encryption, user authentication, retention controls, access logs, consent, secure deletion, bias monitoring, and a clear incident-response process. The current prototype must not be used as an autonomous legal or financial decision maker.

Chapter Summary

DocSaarthi can improve information accessibility when used as a transparent, source-linked assistant. Its benefits depend on privacy protection, careful limitation of claims, and continued human oversight.

Chapter 13

Project Management and Team Contribution

13.1 Introduction

The project was organized as a sequence of requirements, data, design, implementation, evaluation, and reporting milestones. Work was divided between Padmaja Shinde and Akshata Naik while key integration and validation tasks were completed jointly.

13.2 Work Distribution

Table 13.1: Review 3 Work Distribution

Area	Padmaja Shinde	Akshata Naik
Core pipeline	PDF processing, preprocessing, chunking, retrieval, integration	Functional verification and bilingual workflow validation
Language support	Translation integration and intent handling	Translation fidelity and English/Hindi query testing
Quality	Backend debugging and source-grounding tests	Output, page evidence, classification, and unsupported-query tests
Documentation	Technical implementation content	Test evidence, results review, and presentation support

13.3 Project Timeline and Milestones

Phase	Milestone	Deliverable
Review 1	Problem definition, literature, dataset planning	Chapters 1–5 and proposed architecture
Review 2	Prototype implementation and experimentation	Chapters 6–8, application modules, preliminary tests
Review 3	Evaluation, error analysis, final results, impact, conclusion	Chapters 9–14, evidence appendices, final PDF and source archive

13.4 Team Coordination

The team used task-based ownership, shared review of interfaces between modules, and joint end-to-end testing. Findings from testing were converted into implementation changes and regression cases so that fixes could be verified after later edits.

13.5 Testing and Quality Assurance

Quality assurance includes automated unit and integration tests, an eight-document bilingual holdout, source-excerpt checks, value-preservation checks, and manual inspection of representative outputs. The complete automated suite was rerun for Review 3 and passed 18/18 tests.

13.6 Version / Progress Tracking

Source code and report material are maintained as version-controlled project files. Progress is tracked through implementation modules, test scripts, generated evaluation samples, and successive Review 1, Review 2, and Review 3 report chapters. Sensitive configuration values are separated from source through environment settings.

13.7 Individual Contribution Summary

Padmaja's primary contribution is the core document and NLP processing pipeline. Akshata's primary contribution is bilingual interaction, validation, testing, and system-level verification. Both members contributed to debugging, integration, documentation, and the final evaluation narrative. Detailed evidence is provided in Chapter 8 and Appendix E.

Chapter Summary

The staged plan and complementary team roles enabled the project to progress from design to a tested final prototype. Traceable tests and artifacts provide evidence for the reported contributions.

Chapter 14

Conclusion and Future Scope

14.1 Introduction

This chapter concludes the DocSaarthi project by summarizing the implemented outcome, major findings, current limitations, and practical directions for extension.

14.2 Project Conclusion

DocSaarthi demonstrates an end-to-end bilingual assistant for text-based legal and financial PDFs. It integrates page-level extraction, language and domain detection, structured information extraction, searchable evidence chunks, cross-language question handling, translation, grounded answers, and citations in a web application.

14.3 Key Findings

Page-linked chunking improves traceability; preserving dates, names, percentages, and currency values is essential for trustworthy bilingual output; intent data must be kept separate from document evidence; and explicit abstention is safer than forcing an answer. On the controlled Review 3 evaluation, all 18 tests passed and all eight holdout documents were correctly classified with correct language detection.

14.4 Project Limitations

The benchmark is synthetic and too small for general accuracy claims. The system lacks OCR for scanned pages, durable encrypted storage, authentication, calibrated probabilistic confidence, large-scale retrieval evaluation, and formal external human evaluation. Coverage of languages and document types is limited, and the local translation memory cannot translate arbitrary unseen passages.

14.5 Future Scope

Future work should include licensed real-world evaluation data, OCR with layout preservation, multilingual embeddings or a vector index, calibrated classification, retrieval ranking experiments, secure user-scoped storage, authentication and audit logs, formal human evaluation, support for additional Indian languages, and deployment monitoring. A carefully evaluated domain model may be introduced only after representative data and privacy controls are available.

Chapter Summary

The project meets its academic goal by delivering a functional, transparent, and testable prototype. Its strongest foundation is source grounding; future work should preserve that principle while improving scale, coverage, privacy, and measured quality.

Bibliography

- [1] J. Gala, P. A. Chitale, A. K. Raghavan, V. Gumma, S. Doddapaneni, A. K. M, J. A. Nawale, A. Sujatha, R. Puduppully, V. Raghavan, P. Kumar, M. M. Khapra, R. Dabre, and A. Kunchukuttan, "Indictrans2: Towards high-quality and accessible machine translation models for all 22 scheduled indian languages," *Transactions on Machine Learning Research*, 2023.
- [2] S. Khanuja, D. Bansal, S. Mehtani, S. Khosla, A. Dey, B. Gopalan, D. K. Margam, P. Aggarwal, R. T. Nagipogu, S. Dave, S. Gupta, S. C. B. Gali, V. Subramanian, and P. Talukdar, "Muril: Multilingual representations for indian languages," 2021.
- [3] V. Malik, R. Sanjay, S. K. Nigam, K. Ghosh, S. K. Guha, A. Bhattacharya, and A. Modi, "Ildc for cjpe: Indian legal documents corpus for court judgment prediction and explanation," in *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pp. 4046–4062, Association for Computational Linguistics, 2021.
- [4] D. T. Araci, "Finbert: Financial sentiment analysis with pre-trained language models," 2019.
- [5] P. Kalamkar, A. Agarwal, A. Tiwari, S. Gupta, S. Karn, and V. Raghavan, "Named entity recognition in indian court judgments," in *Proceedings of the Natural Legal Language Processing Workshop 2022, (Abu Dhabi, United Arab Emirates (Hybrid))*, pp. 184–193, Association for Computational Linguistics, 2022.
- [6] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pp. 4171–4186, 2019.

Appendix A

AI Compliance

A.1 AI Tools Used

The following Artificial Intelligence (AI) tools were used during the preparation of this micro project report:

- ChatGPT (OpenAI)
- Codex (OpenAI)
- Claude AI (Anthropic)

These tools were used only as academic writing assistants to improve the quality and presentation of the report.

A.2 Nature of Assistance

The AI tools were used for the following purposes:

- Improving grammar, sentence structure, and academic writing style.
- Assisting in the organization and formatting of the report in $\text{\LaTeX}$ (Overleaf).
- Providing explanations of NLP concepts, machine learning models, and research methodologies.
- Assisting in drafting sections of the report based on the project requirements.
- Helping summarize and understand selected research papers.
- Suggesting suitable datasets, preprocessing techniques, feature representation methods, and evaluation metrics relevant to the proposed NLP system.

- Assisting in generating diagrams, workflow descriptions, and documentation while ensuring consistency throughout the report.
- Assisting in reorganizing the Review 3 report, preparing evaluation tables from verified test output, and checking the compiled PDF layout.

The final selection of content, project design, implementation decisions, and verification of technical information were carried out by the author.

A.3 Ethical Compliance Statement

I hereby declare that Artificial Intelligence (AI) tools were used only to assist in academic writing, report organization, language improvement, and understanding technical concepts. All project ideas, implementation decisions, dataset preparation, literature review, analysis, and final conclusions were carefully reviewed and verified by me. Proper credit has been given wherever external sources were consulted, and all research papers have been cited appropriately. I affirm that this report represents my own academic work and complies with the ethical guidelines prescribed by the institution regarding the responsible use of AI-assisted tools.

Appendix B

Plagiarism Report

The plagiarism report generated for the Stage-1 Micro Project is attached in the following pages.



DocSaarthi_Report_Complete (3).pdf

 SAKEC - Shah and Anchor Kutchhi Engineering College

Document Details

Submission ID

trn:oid:::3618:153703948

Submission Date

Oct 5, 2026, 11:51 AM GMT+5:30

Download Date

Oct 5, 2026, 12:09 PM GMT+5:30

File Name

DocSaarthi_Report_Complete (3).pdf

File Size

3.5 MB

115 Pages

21,370 Words

132,491 Characters



6% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

-  **203 Not Cited or Quoted 6%**
Matches with neither in-text citation nor quotation marks
-  **2 Missing Quotations 0%**
Matches that are still very similar to source material
-  **2 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 6%  Internet sources
- 4%  Publications
- 8%  Submitted works (Student Papers)

Integrity Flags

1 Integrity Flag for Review

-  **Hidden Text**
39 suspect characters on 2 pages
Text is altered to blend into the white background of the document.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Appendix C

AI Writing Report



DocSaarthi_Report_Complete (3).pdf

SAKEC - Shah and Anchor Kutchhi Engineering College

Document Details

Submission ID
trn:oid:::3618:153703948

Submission Date
Oct 5, 2026, 11:51 AM GMT+5:30

Download Date
Oct 5, 2026, 12:09 PM GMT+5:30

File Name
DocSaarthi_Report_Complete (3).pdf

File Size
3.5 MB

115 Pages

21,370 Words

132,491 Characters





5% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely revised by AI.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.



13 AI-generated 5%

Likely generated or likely generated and revised by AI.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (it may misidentify writing that is likely human generated as AI generated and likely AI generated as human generated) so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.



Appendix D

Assessment Rubrics

Assessment Criterion	Level 4 (Excellent)	Level 3 (Good)	Level 2 (Satisfactory)	Level 1 (Needs Improvement)
Problem Understanding & Objectives (IA) 3%	Clearly explains the problem, motivation, objectives, scope, assumptions, and constraints with strong understanding.	Explains the problem and objectives with minor omissions.	Basic understanding; lacks clarity in objectives or justification.	Unable to explain the problem or objectives satisfactorily.
Literature Review (Minimum 3 Papers per Student) (IA) 5%	Reviews 3 or more assigned research papers critically, compares approaches, identifies research gaps, and explains relevance to the project.	Reviews assigned papers with adequate comparison and identifies basic research gaps.	Reviews papers descriptively with limited comparison or weak research gap analysis.	Incomplete review or unable to explain assigned papers.
Dataset & Data Preparation (IA) 4%	Clearly explains dataset selection, characteristics, preprocessing decisions, and justifies the chosen techniques.	Good explanation with minor omissions in justification or preprocessing.	Basic explanation with limited understanding of preprocessing decisions.	Unable to justify dataset selection or preprocessing techniques.
Technical Understanding & Communication (IA) 3%	Demonstrates excellent understanding of the proposed NLP pipeline and communicates ideas confidently during viva.	Good understanding with minor gaps in explanation.	Partial understanding; requires significant prompting.	Unable to explain technical concepts or contribution.

Dataset Collection & Documentation (GA) 2%	Dataset is appropriate, well documented, justified, and includes source, characteristics, licensing, and annotation details (if applicable).	Dataset is suitable with minor omissions.	Dataset documentation is incomplete or weakly justified.	Dataset selection/documentation is inadequate.
Data Preparation & EDA (GA) 2%	Appropriate preprocessing workflow and EDA are well designed, justified, and documented.	Most preprocessing steps are appropriate with adequate documentation.	Basic preprocessing and limited EDA.	Preprocessing workflow is incomplete or incorrect.
NLP Pipeline Design (GA) 4%	Comprehensive architecture, workflow, feature representation, model selection, and proposed evaluation strategy are well justified.	Pipeline is complete with minor gaps in justification.	Basic pipeline with limited justification.	Pipeline is incomplete or missing major components.
Report Quality & Team Coordination (GA) 2%	Report is well organized, technically sound, professionally formatted, and demonstrates effective collaboration.	Report is complete with minor formatting or coordination issues.	Report is adequate but lacks consistency or organization.	Poorly organized report with significant omissions.

Appendix E

Supporting Evidence

E.1 Dataset Samples

The project dataset contains 400 Hindi intent records. The verified distribution is: Agreement 40, Payment 20, Court Proceeding 80, Confidentiality 40, General 60, Finance 100, Loan 20, Tax 20, and Insurance 20. Records are used for intent support only and not as document evidence.

E.2 Preprocessing Examples

Verified preprocessing cases include repair of sentence-breaking PDF line breaks, preservation of Devanagari combining marks, Unicode normalization, and page-linked evidence chunking. Original source text is retained for verification.

E.3 Experimental Logs

The final automated run reports 18 tests, 18 passes, and zero failures. The classifier evaluation reports eight labelled samples, 100% classification accuracy, macro-F1 1.00, and 100% language-detection accuracy. These results apply only to the controlled synthetic benchmark.

E.4 Additional Results

Additional validated behaviors include English-to-Hindi retrieval, Hindi-to-English retrieval, preservation of ₹ amounts, percentages, dates, and names, safe handling of unavailable translation, and abstention when a requested fact is absent.

E.5 Individual Contribution Evidence

Evidence includes source modules, test cases, evaluation scripts, bilingual sample PDFs, result logs, interface screenshots, and report chapters. Chapter 8 records task-level contributions and Chapter 13 summarizes Review 3 responsibilities.